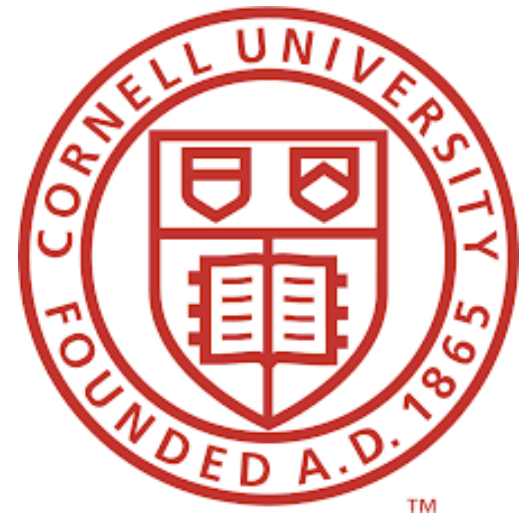


AUTOMATON-BASED EXPRESSIVE MONITORING OF BLACKBOX SYSTEMS

BITS Pilani
Aug 31, 2026

Karuna Grewal, Brighten Godfrey, Justin Hsu



UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN

REAL-WORLD SOFTWARE TO BE VERIFIED IS OFTEN BLACKBOX



Without access to code, we are required to verify properties like:

Data-Compliance:

Development:

DevOps/Testing:

REAL-WORLD SOFTWARE TO BE VERIFIED IS OFTEN BLACKBOX



Without access to code, we are required to verify properties like:

Data-Compliance: enforce data-protection regulations, like GDPR, HIPAA, MiFID II

Development:

DevOps/Testing:

REAL-WORLD SOFTWARE TO BE VERIFIED IS OFTEN BLACKBOX



Without access to code, we are required to verify properties like:

Data-Compliance: enforce data-protection regulations, like GDPR, HIPAA, MiFID II

Development: always log critical alerts in your vehicle

DevOps/Testing:

REAL-WORLD SOFTWARE TO BE VERIFIED IS OFTEN BLACKBOX



Without access to code, we are required to verify properties like:

Data-Compliance: enforce data-protection regulations, like GDPR, HIPAA, MiFID II

Development: always log critical alerts in your vehicle

DevOps/Testing: use canary version of a new algorithm

REAL-WORLD SOFTWARE TO BE VERIFIED IS OFTEN BLACKBOX



⚠ Mishandling data can lead to regulatory penalties and loss of client trust!

Presently,
Ad-hoc software-based implementation of safety checks

We need to streamline and automate policy enforcement

Presently,
Ad-hoc software-based implementation of safety checks

We need to streamline and automate policy enforcement



(frequently evolving)



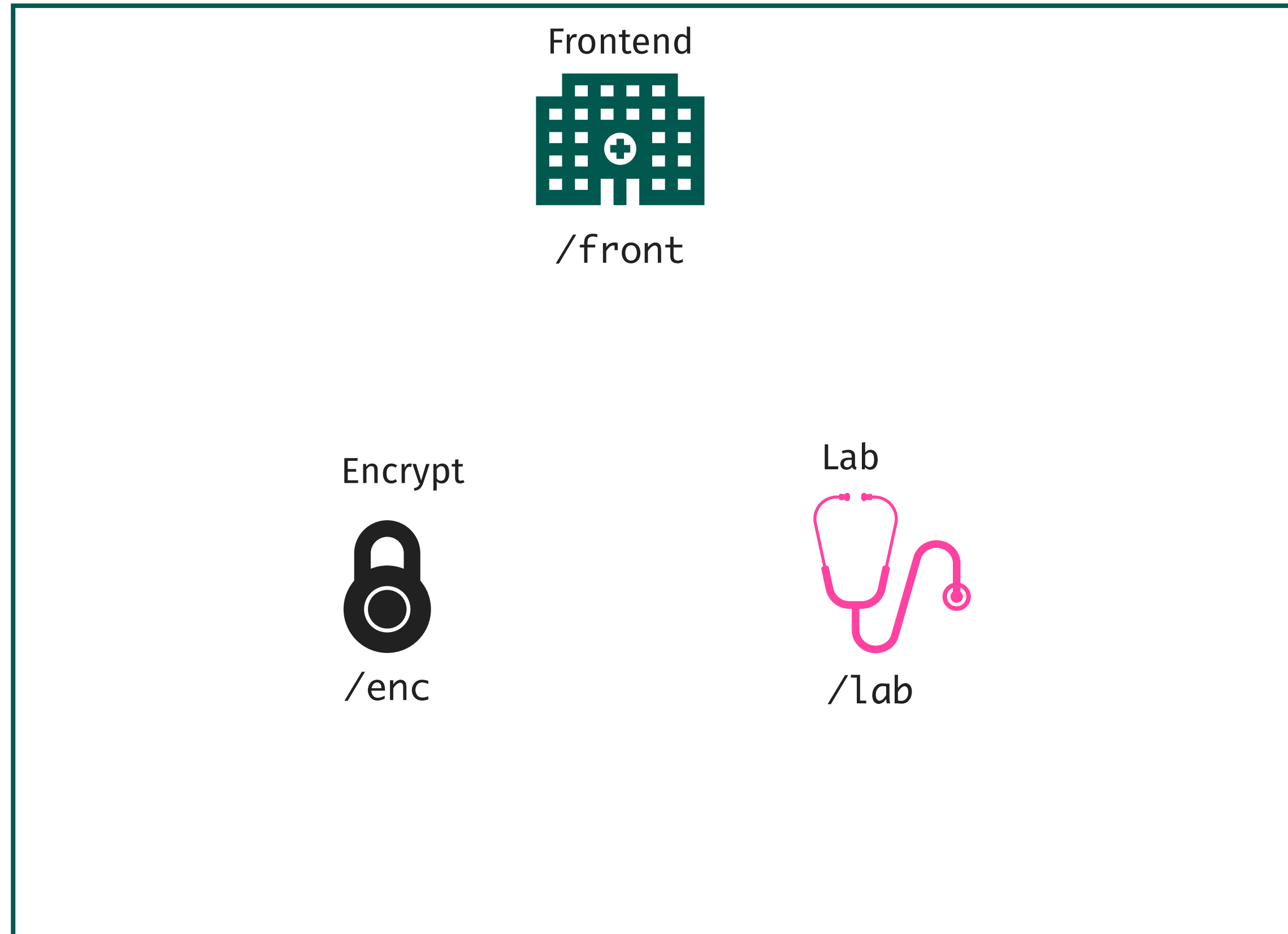
Huge semantic gap between safety specification and their code-level implementation

Goal:

Specify and verify correct behavior of blackbox system

*Requirements: **no source code**; **non-invasive** enforcement*

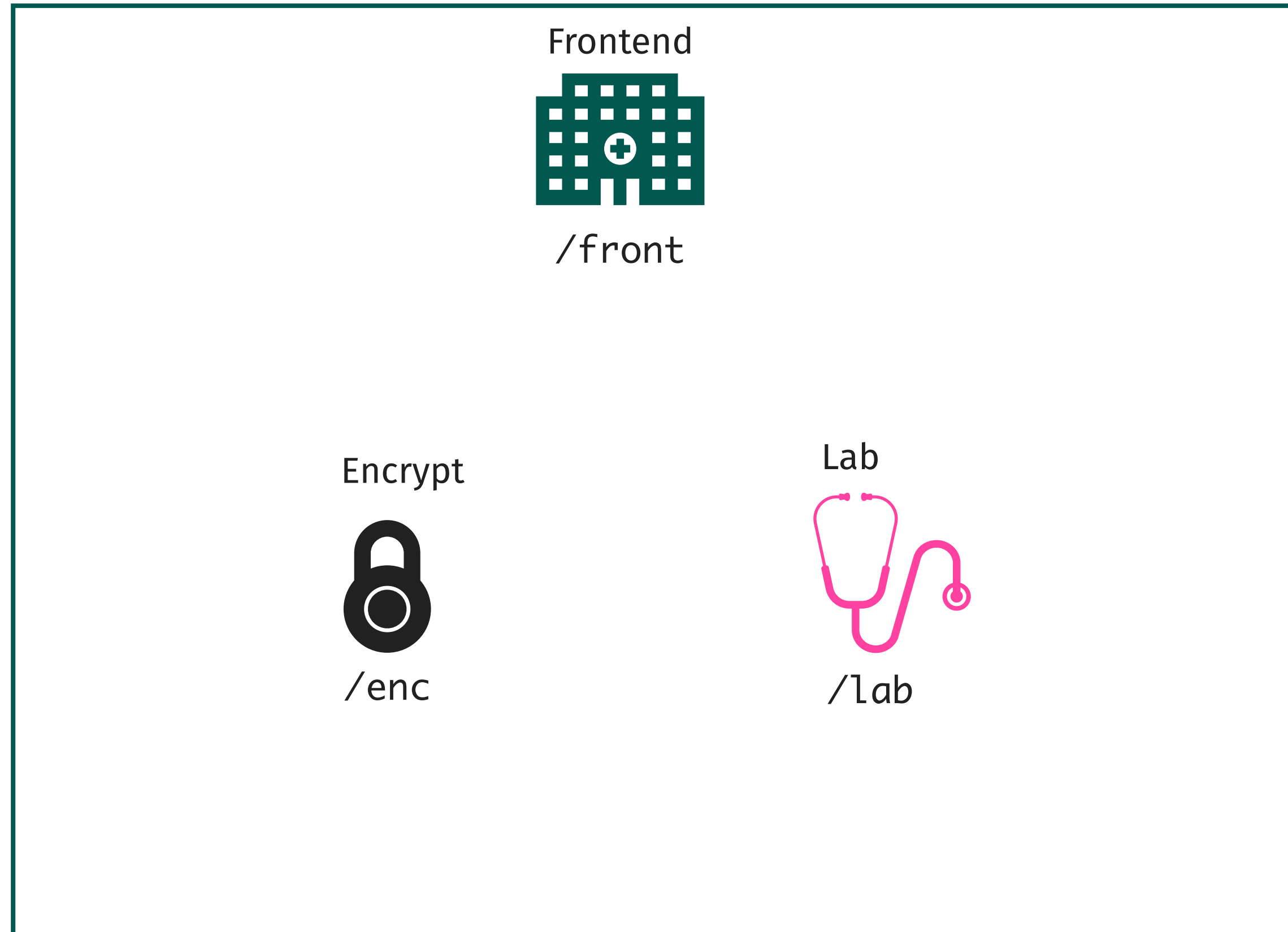
RUNNING EXAMPLE: BLACKBOX HEALTHCARE APP IN THE CLOUD



To request a medical test from an external lab

We model blackbox systems in terms of components, interface, network protocol

RUNNING EXAMPLE: BLACKBOX HEALTHCARE APP IN THE CLOUD

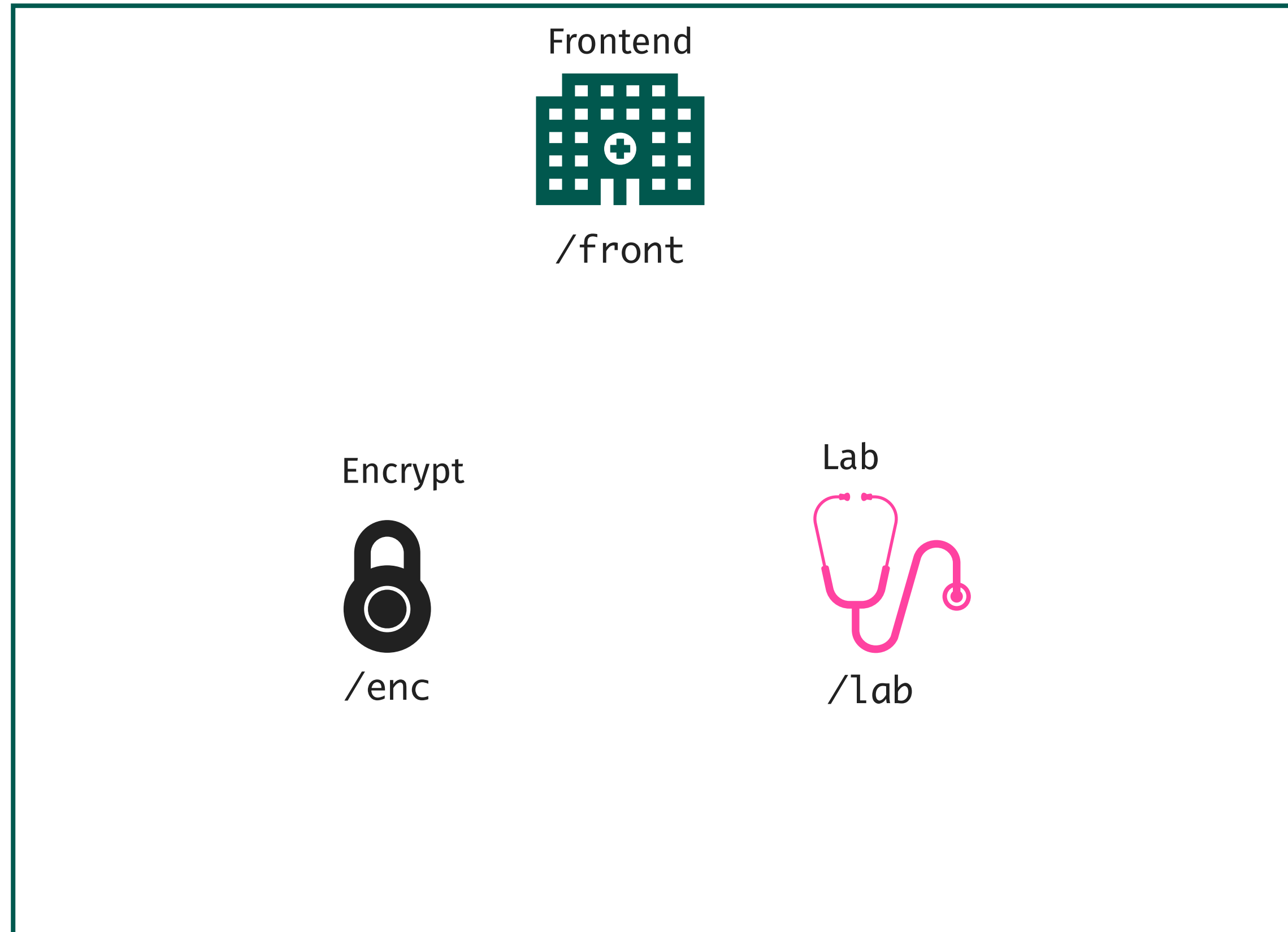


To request a medical test from an external lab

Business logic decomposed across:
components $\equiv$ **services**

We model blackbox systems in terms of components, interface, network protocol

RUNNING EXAMPLE: BLACKBOX HEALTHCARE APP IN THE CLOUD



To request a medical test from an external lab

Business logic decomposed across:

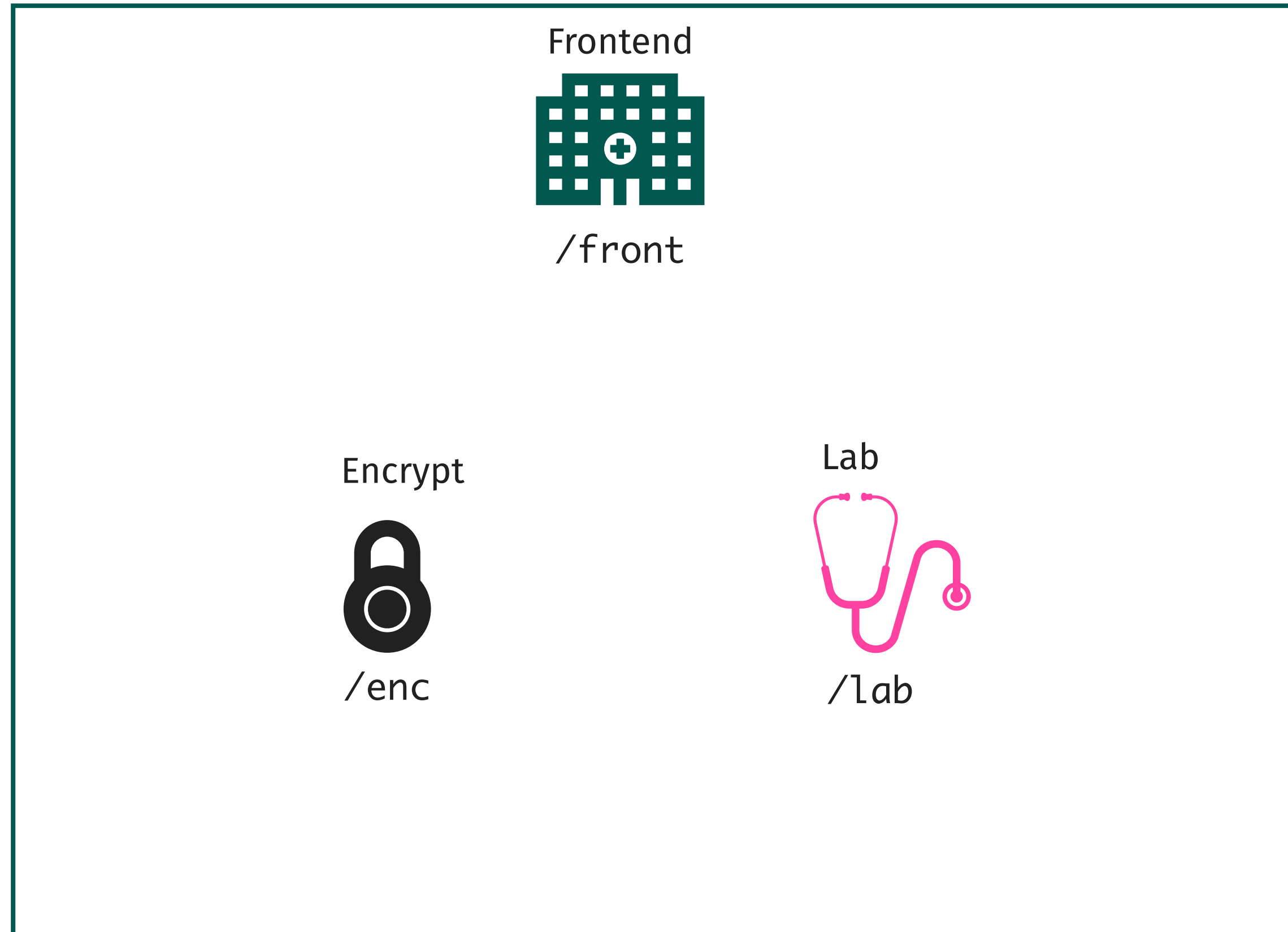
components $\equiv$ **services**

that interact through

inter-component communication interface $\equiv$ **APIs**

We model blackbox systems in terms of components, interface, network protocol

RUNNING EXAMPLE: BLACKBOX HEALTHCARE APP IN THE CLOUD



To request a medical test from an external lab

Business logic decomposed across:

components $\equiv$ **services**

that interact through

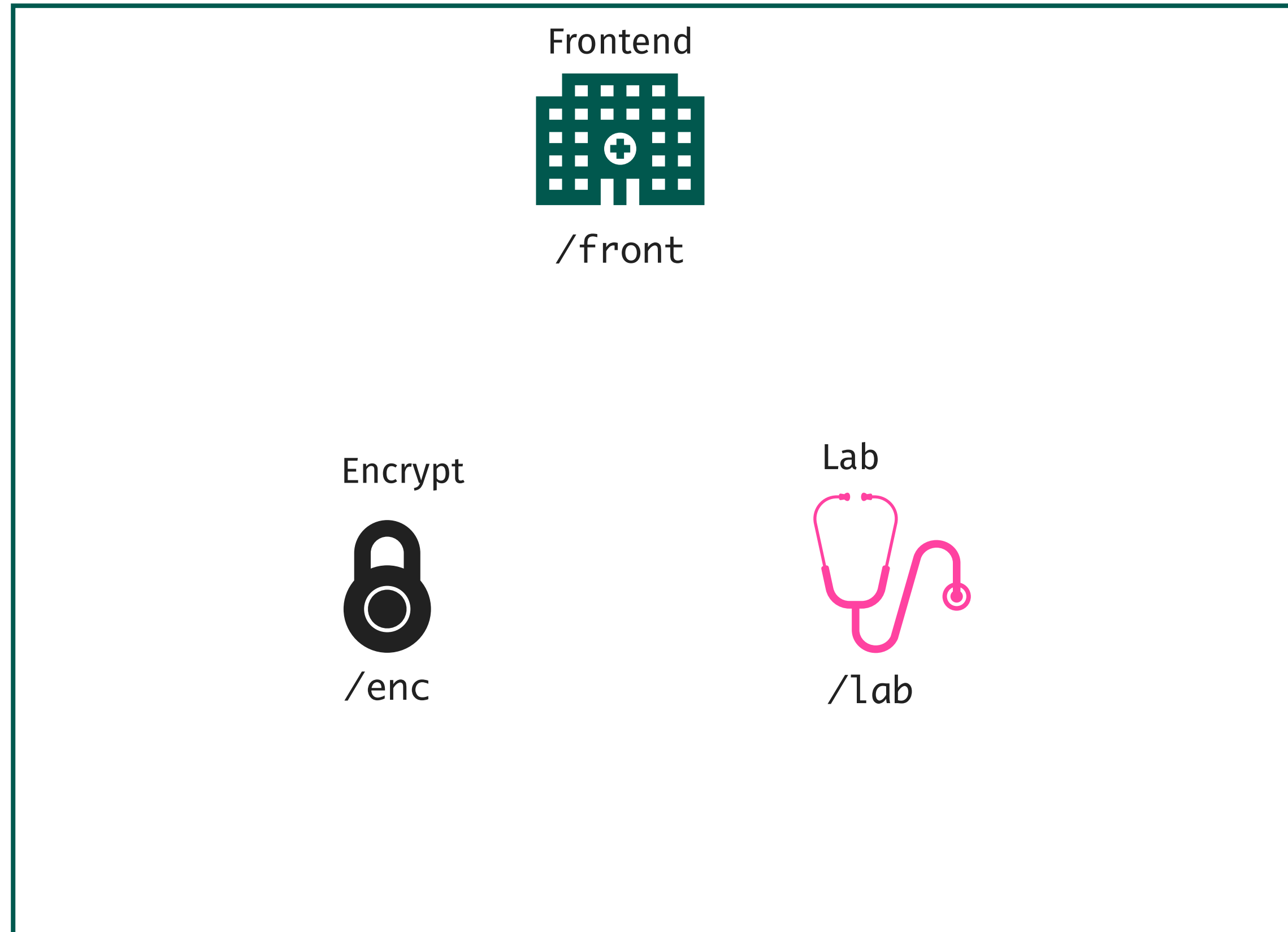
inter-component communication interface $\equiv$ **APIs**

over

network protocol $\equiv$ **HTTP**

We model blackbox systems in terms of components, interface, network protocol

RUNNING EXAMPLE: BLACKBOX HEALTHCARE APP IN THE CLOUD



To request a medical test from an external lab

Business logic decomposed across:

components $\equiv$ **services**

that interact through

inter-component communication interface $\equiv$ **APIs**

over

network protocol $\equiv$ **HTTP**

(implemented using microservice design paradigm)

We model blackbox systems in terms of components, interface, network protocol

OUR WORK

Which components can talk and in what order? + how should the data flow across the components?

OOPSLA'25



SafeTree: Expressive Tree Policies for Microservices*

[KARUNA GREWAL](#), Cornell University, USA

[BRIGHTEN GODFREY](#), University of Illinois at Urbana-Champaign, USA

[JUSTIN HSU](#), Cornell University, USA

Under Submission

SafeNom: Data-Aware Policies for Microservices

Karuna Grewal, Brighten Godfrey, Justin Hsu

OUR WORK

Which components can talk and in what order? + how should the data flow across the components?

OOPSLA'25



SafeTree: Expressive Tree Policies for Microservices*

KARUNA GREWAL, Cornell University, USA

BRIGHTEN GODFREY, University of Illinois at Urbana-Champaign, USA

JUSTIN HSU, Cornell University, USA

Under Submission

SafeNom: Data-Aware Policies for Microservices

Karuna Grewal, Brighten Godfrey, Justin Hsu

OUR WORK

Which components can talk and in what order? + how should the data flow across the components?

OOPSLA'25



SafeTree: Expressive Tree Policies for Microservices*

KARUNA GREWAL, Cornell University, USA

BRIGHTEN GODFREY, University of Illinois at Urbana-Champaign, USA

JUSTIN HSU, Cornell University, USA

Under Submission

SafeNom: Data-Aware Policies for Microservices

Karuna Grewal, Brighten Godfrey, Justin Hsu



Specification language



Automata-based Runtime Monitor



Non-invasive implementation

OUR WORK

Which components can talk and in what order? + how should the data flow across the components?

OOPSLA'25



SafeTree: Expressive Tree Policies for Microservices*

KARUNA GREWAL, Cornell University, USA

BRIGHTEN GODFREY, University of Illinois at Urbana-Champaign, USA

JUSTIN HSU, Cornell University, USA

Under Submission

SafeNom: Data-Aware Policies for Microservices

Karuna Grewal, Brighten Godfrey, Justin Hsu

Insight: control flow trees as nested words



Specification language



Automata-based Runtime Monitor



Non-invasive implementation

Specification: SafeTree Language



EXAMPLE: ENCRYPTION PROPERTY

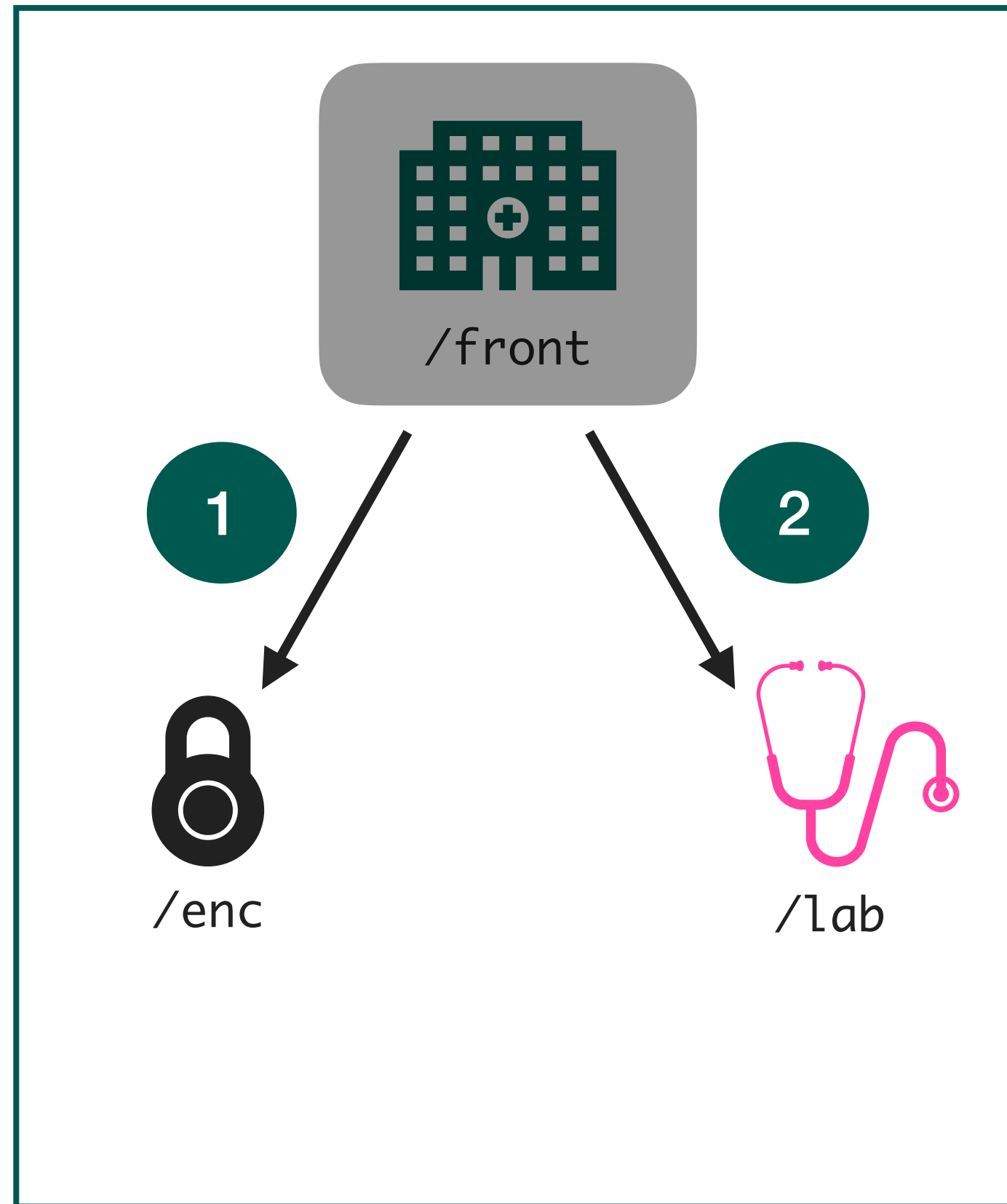
Purpose: to prevent leaking personal health information

Example: encrypt health record information before sharing with external lab

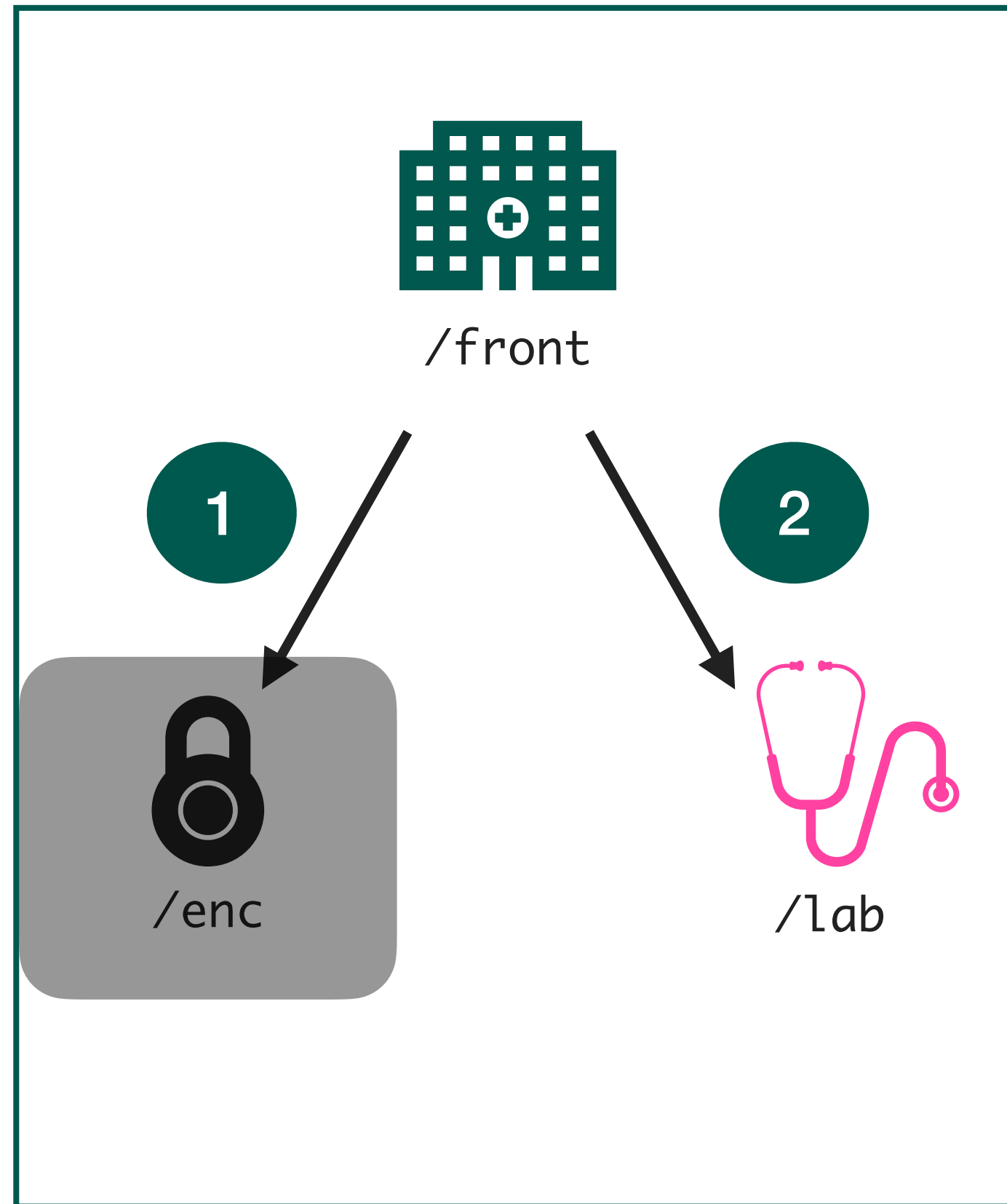


Example valid execution trace of a lab request

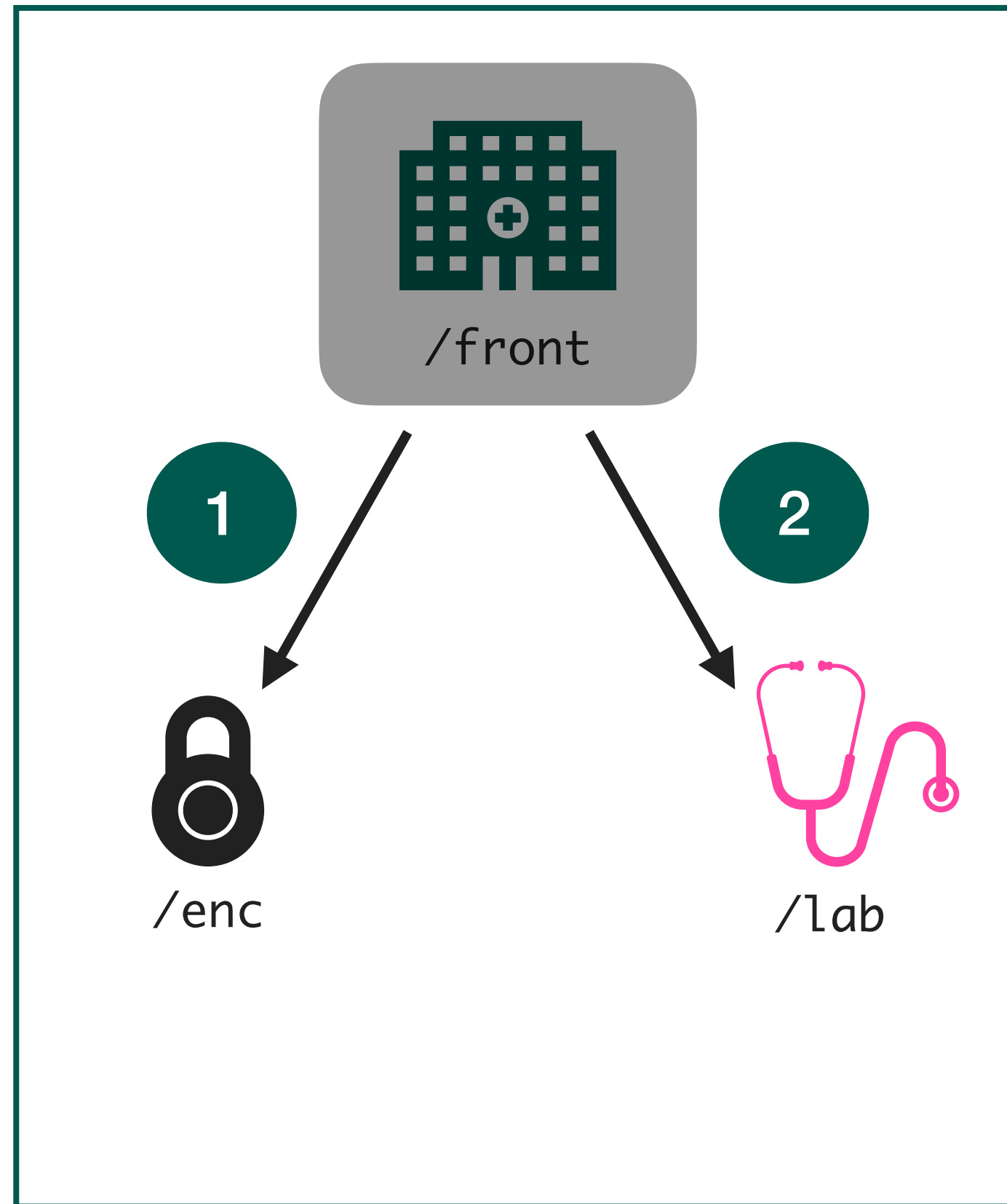
A VALID EXECUTION TRACE



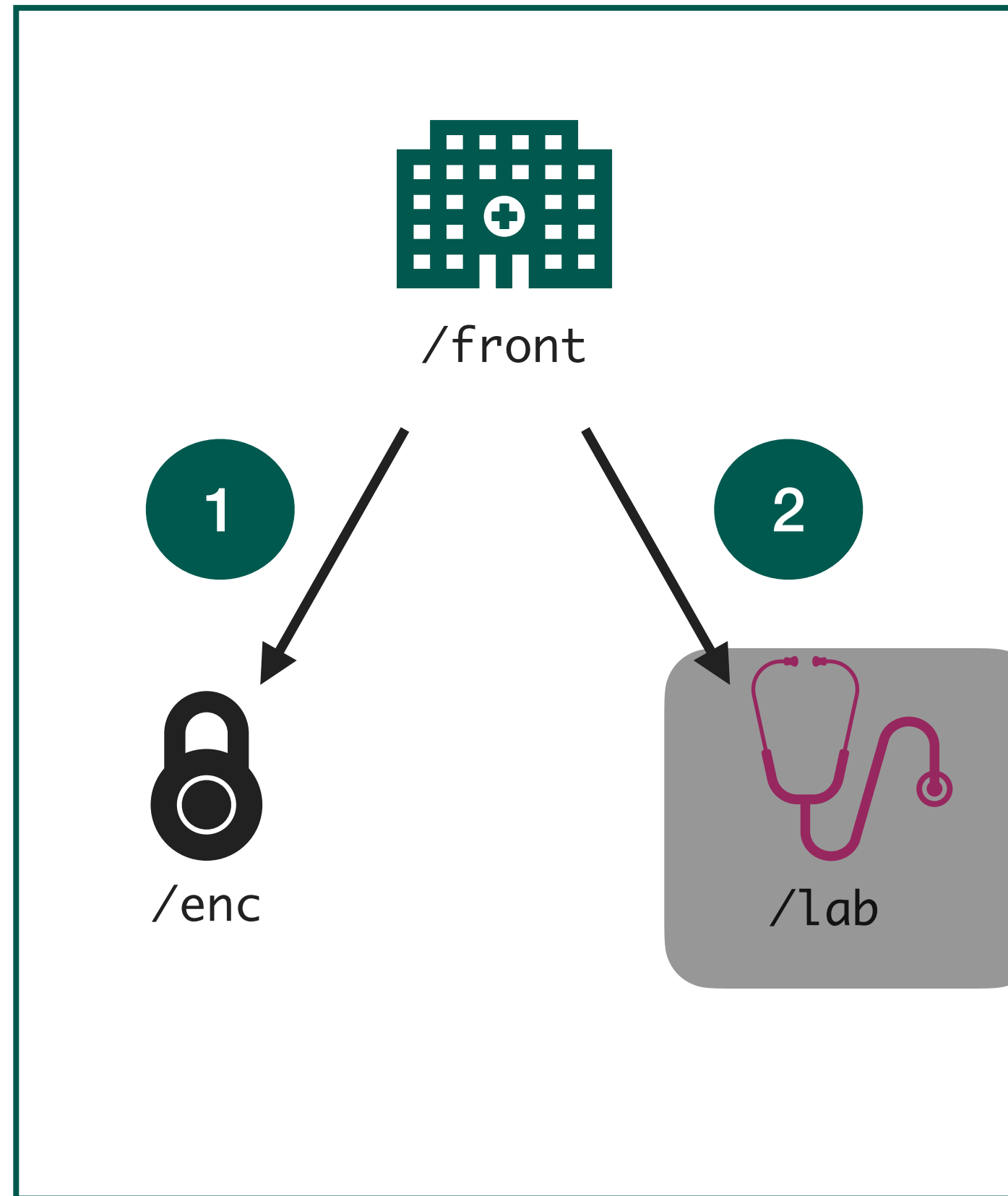
A VALID EXECUTION TRACE



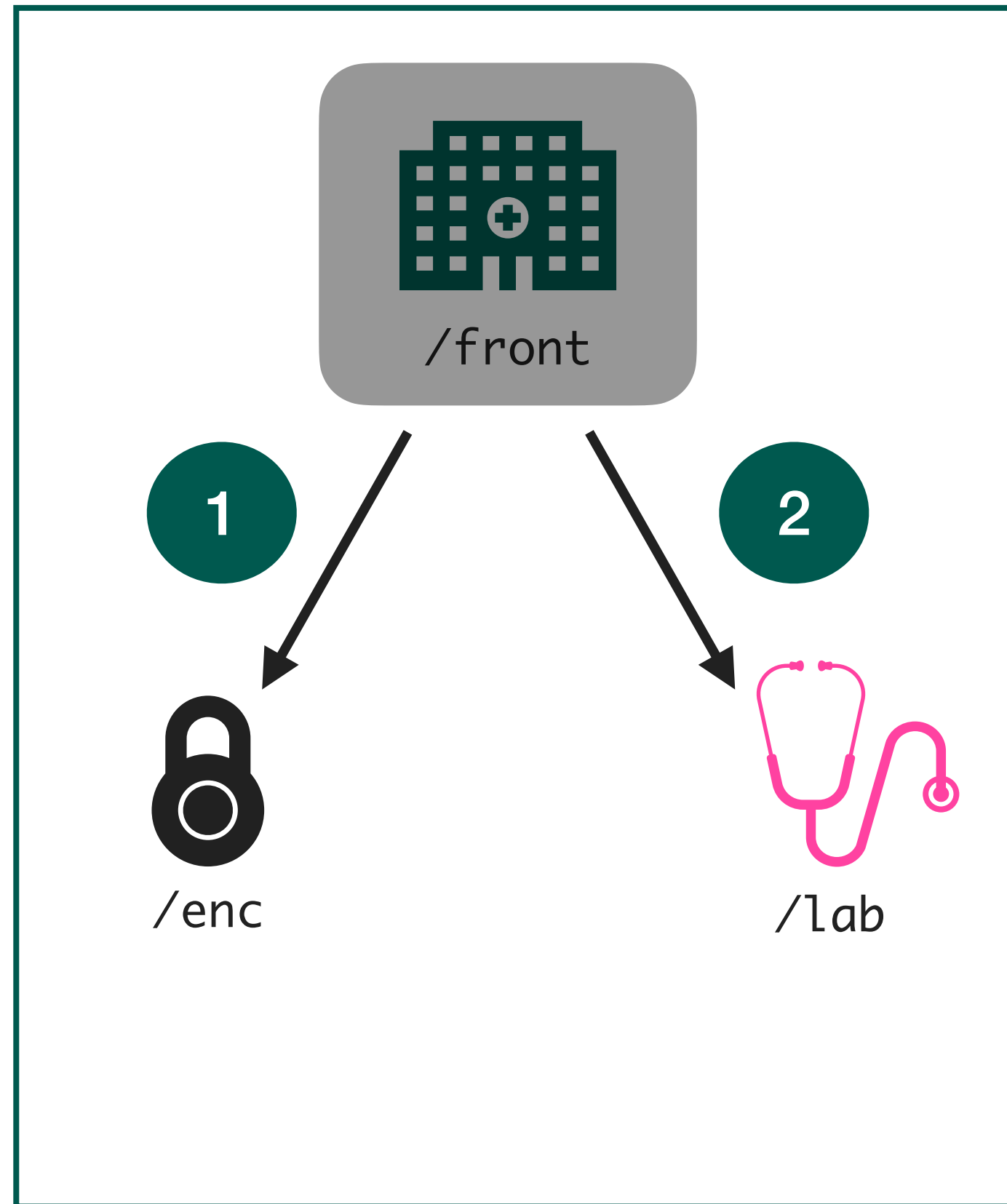
A VALID EXECUTION TRACE



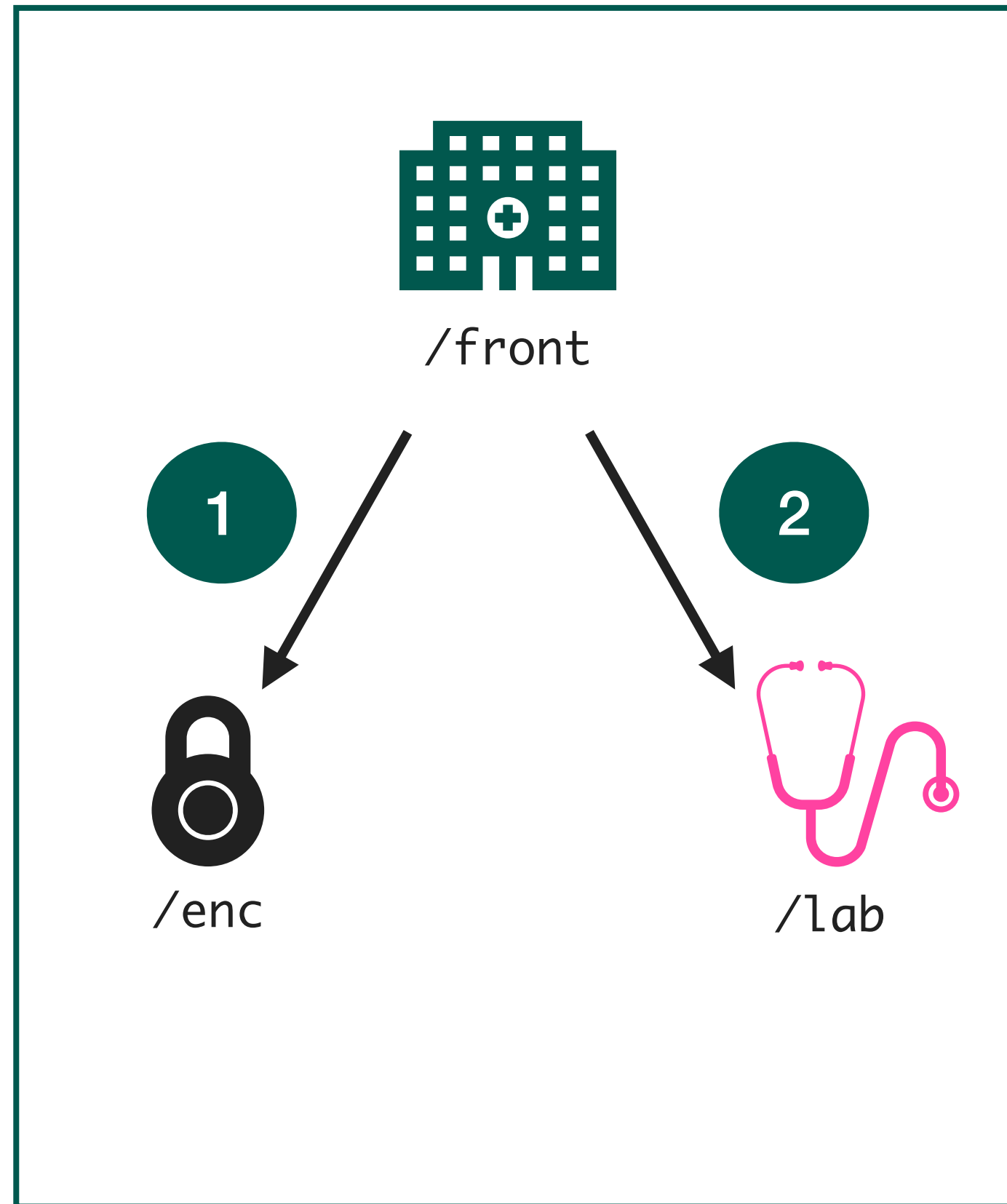
A VALID EXECUTION TRACE



A VALID EXECUTION TRACE

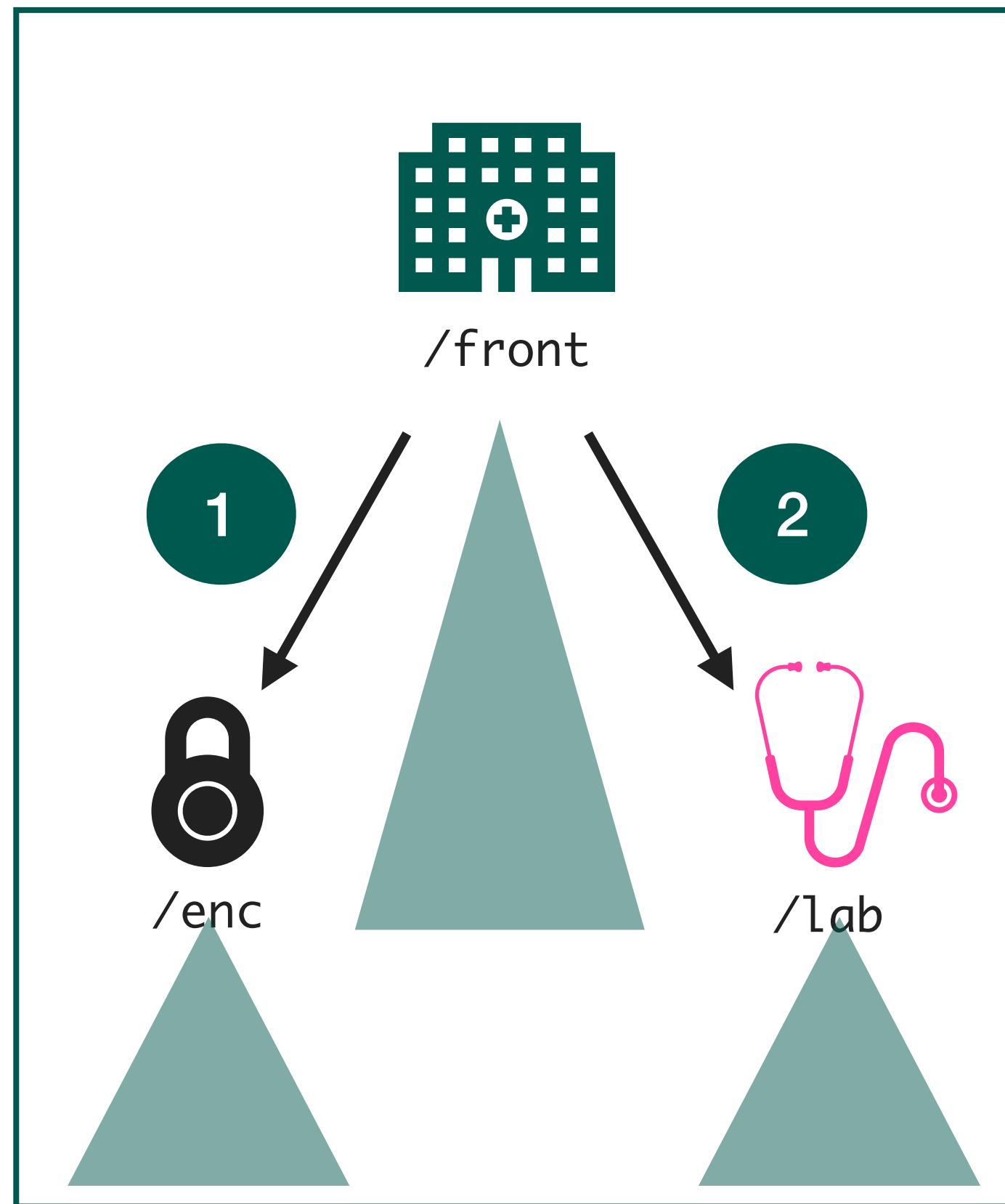


A VALID EXECUTION TRACE



A VALID EXECUTION TRACE

Notice what matters here?



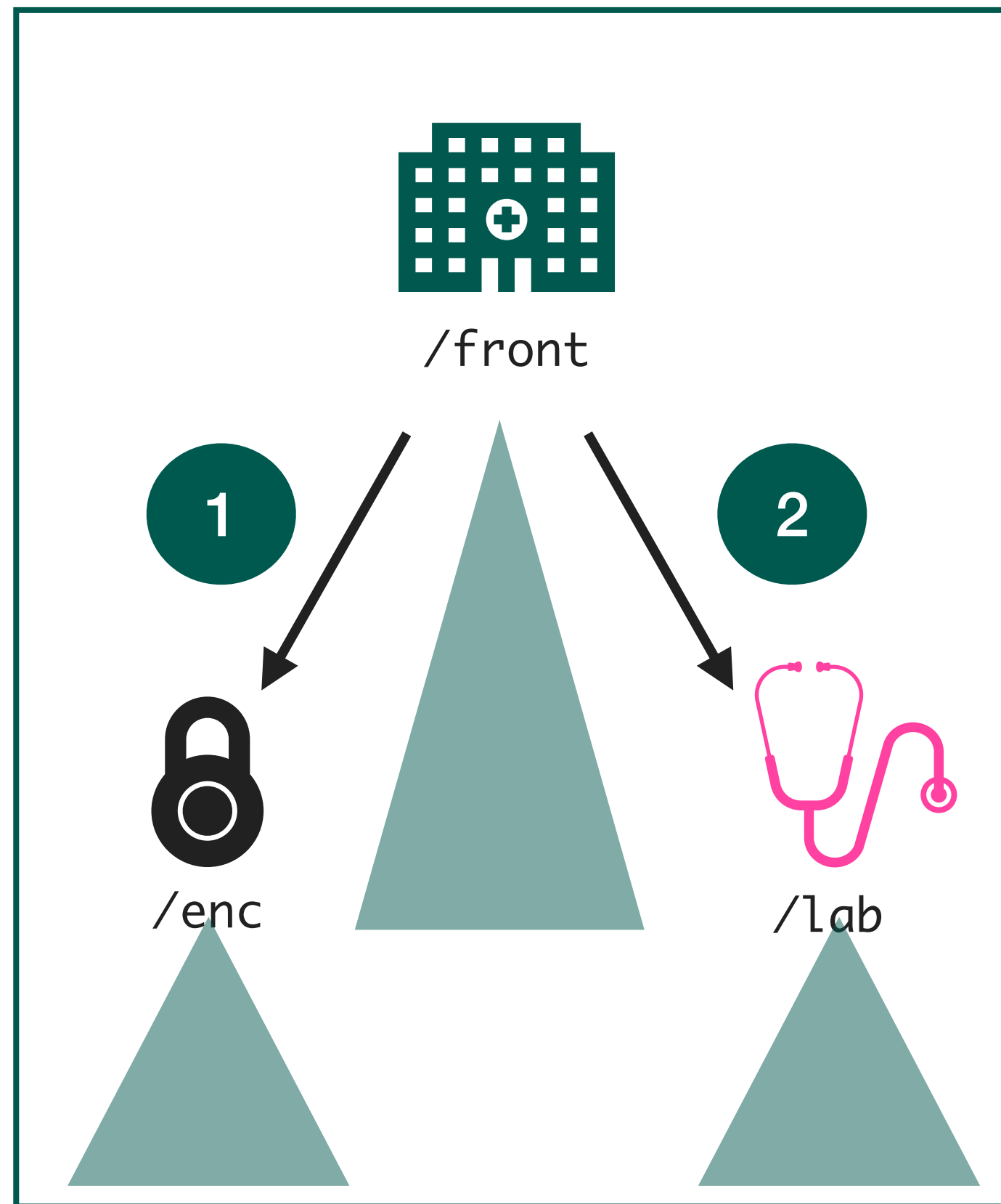
▲ : Further subtree of API calls

A VALID EXECUTION TRACE

Notice what matters here?

Linear ordering of APIs:

/Enc is called before /Lab



▲ : Further subtree of API calls

A VALID EXECUTION TRACE

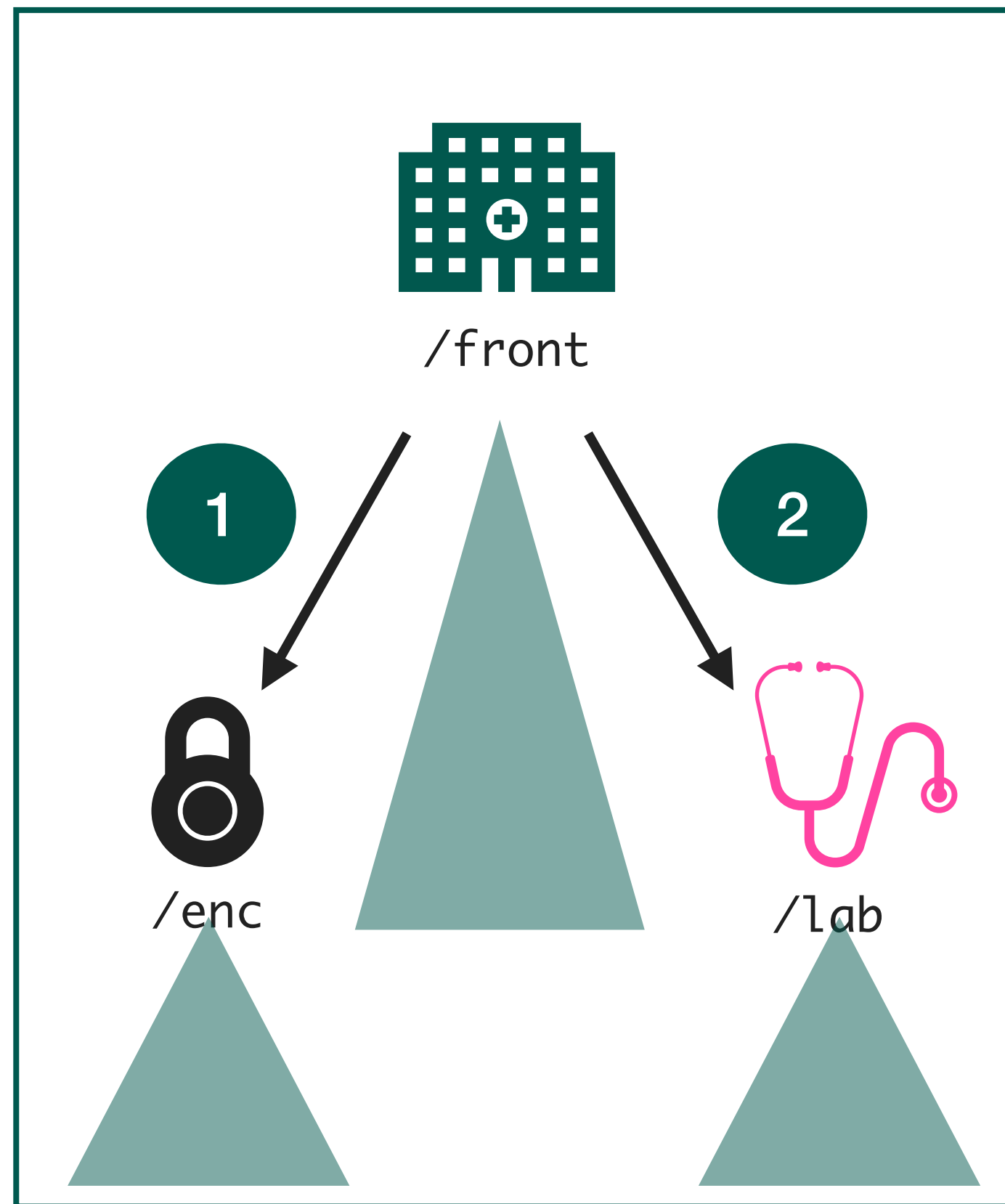
Notice what matters here?

Linear ordering of APIs:

/Enc is called before /Lab

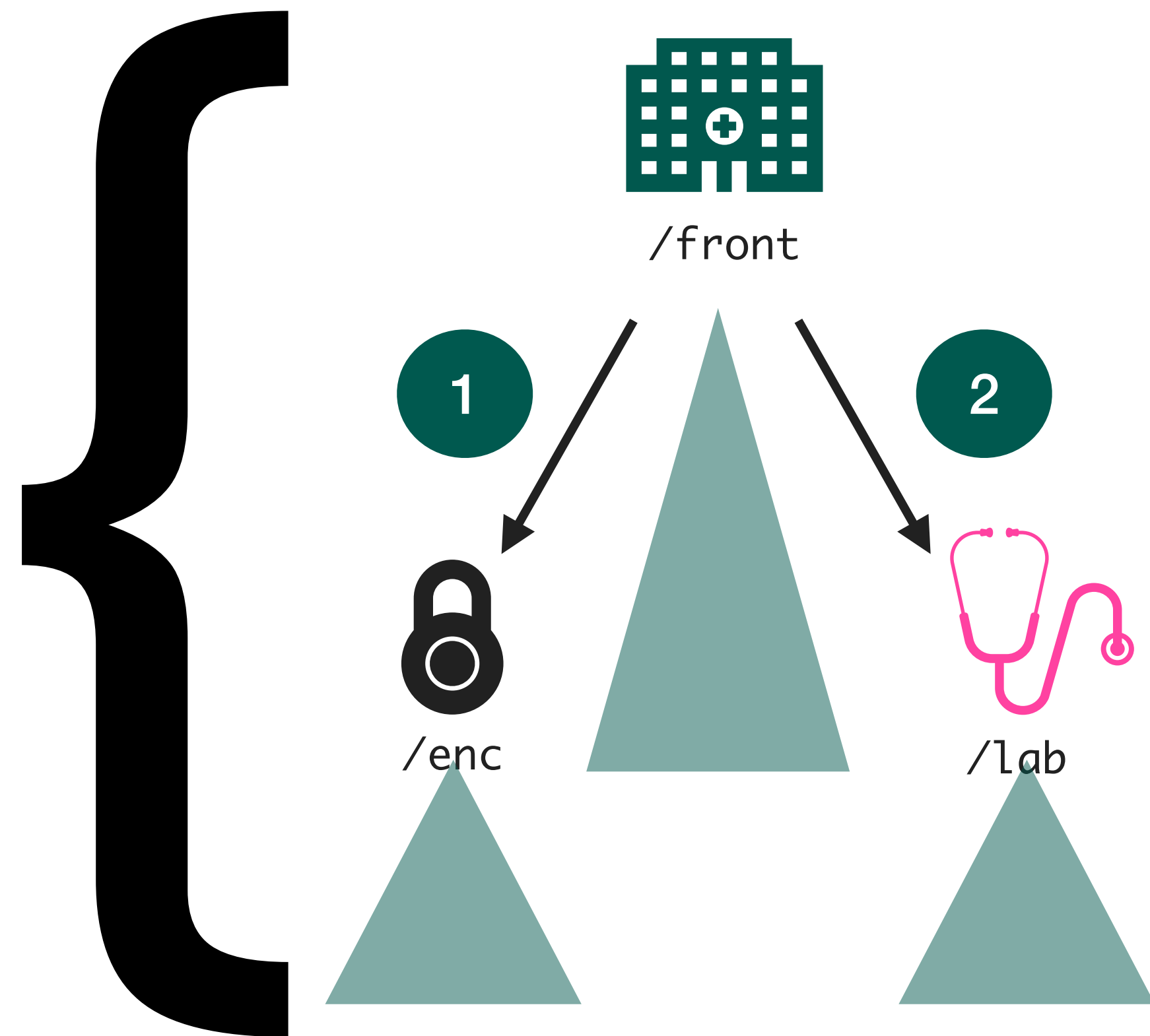
Hierarchical structure of the call tree:

/Enc and /Lab are called by same parent /Front



▲ : Further subtree of API calls

SKELETON OF SAFETREE SPECIFICATION FOR ENCRYPTION PROPERTY



Starting API call is “Front”

At least two subtrees:

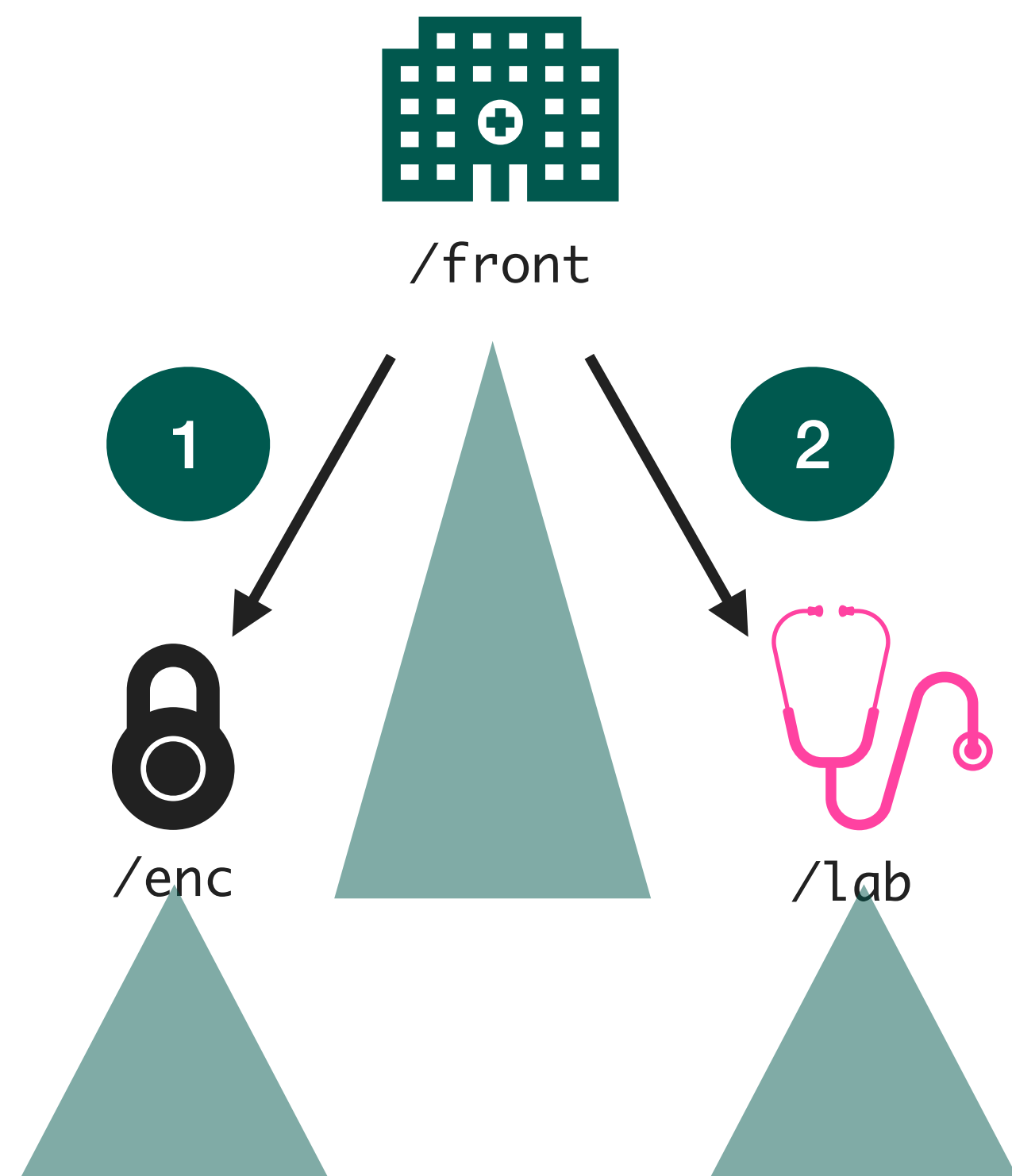
(1) one rooted at “Enc”

(2) one rooted at “Lab”

(1) precedes (2)

(1) and (2) can further call nested APIs

SKELETON OF SAFETREE SPECIFICATION FOR ENCRYPTION PROPERTY



Starting API call is “Front”

At least two subtrees:

(1) one rooted at “Enc”

(2) one rooted at “Lab”

(1) precedes (2)

(1) and (2) can further call nested APIs

=

```
match root=Front  $\xRightarrow{\exists\text{-child}}$  {
```

```
  A subtree with root = /Enc then
```

```
  A subtree with root = /Lab
```

```
}
```

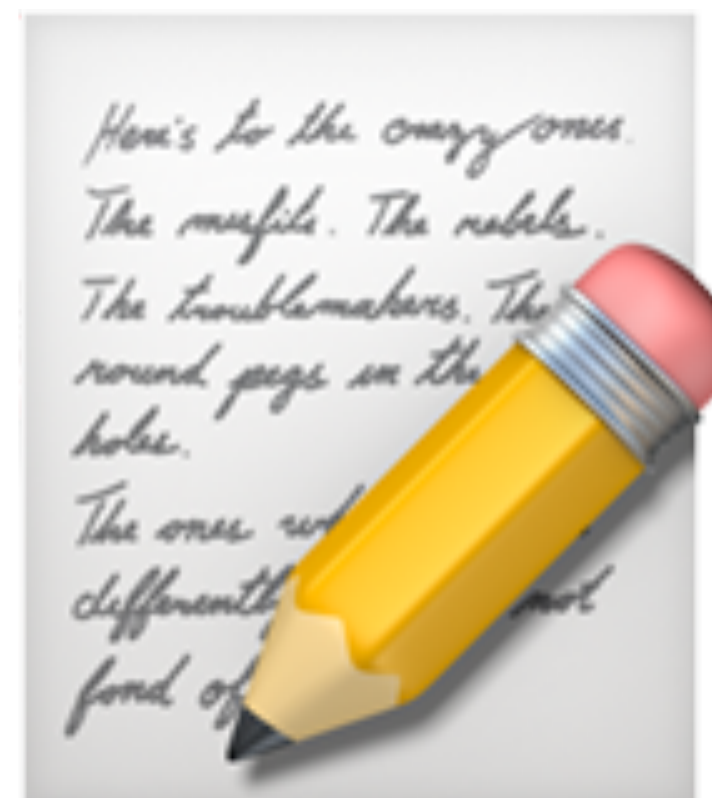

SKELETON OF SAFETREE SPECIFICATION FOR ENCRYPTION PROPERTY

The set of **valid service tree traces** can be modeled as a **nested word language**

=

```
match root=Front  $\xRightarrow{\exists\text{-child}}$  {  
    A subtree with root = /Enc then  
    A subtree with root = /Lab  
}
```

SafeTree Language: Nested Word Underpinnings



Based on Rajeev Alur and P. Madhusudan. 2009. Adding nesting structure to words. J. ACM 56, 3, Article 16 (May 2009)

REFRESHER: A WORD IS A SEQUENCE OF SYMBOLS IN AN ALPHABET

REFRESHER: A WORD IS A SEQUENCE OF SYMBOLS IN AN ALPHABET

Consider APIs = {Front, Enc, Lab , (APIs in microservice app) }

REFRESHER: A WORD IS A SEQUENCE OF SYMBOLS IN AN ALPHABET

Consider APIs = {Front, Enc, Lab, ... (APIs in microservice app) }

Let's start with a “word” over alphabet $\Sigma = \mathcal{C} \uplus \mathcal{R}$

$\mathcal{C} = \{ \text{call-Front, call-Enc, ... (call annotated APIs) } \}$

$\mathcal{R} = \{ \text{ret-Front, ret-Enc, ... (ret annotated APIs) } \}$

REFRESHER: A WORD IS A SEQUENCE OF SYMBOLS IN AN ALPHABET

Consider APIs = {Front, Enc, Lab, ... (APIs in microservice app) }

Let's start with a “word” over alphabet $\Sigma = \mathcal{C} \uplus \mathcal{R}$

$\mathcal{C} = \{ \text{call-Front, call-Enc, ... (call annotated APIs) } \}$

$\mathcal{R} = \{ \text{ret-Front, ret-Enc, ... (ret annotated APIs) } \}$

Example word,

call-Front **call**-Enc **ret**-Enc **call**-Lab **ret**-Lab **ret**-Front

REFRESHER: A WORD IS A SEQUENCE OF SYMBOLS IN AN ALPHABET

Consider APIs = {Front, Enc, Lab, ... (APIs in microservice app) }

Let's start with a “word” over alphabet $\Sigma = \mathcal{C} \uplus \mathcal{R}$

$\mathcal{C} = \{ \text{call-Front, call-Enc, ... (call annotated APIs) } \}$

$\mathcal{R} = \{ \text{ret-Front, ret-Enc, ... (ret annotated APIs) } \}$

Example word,

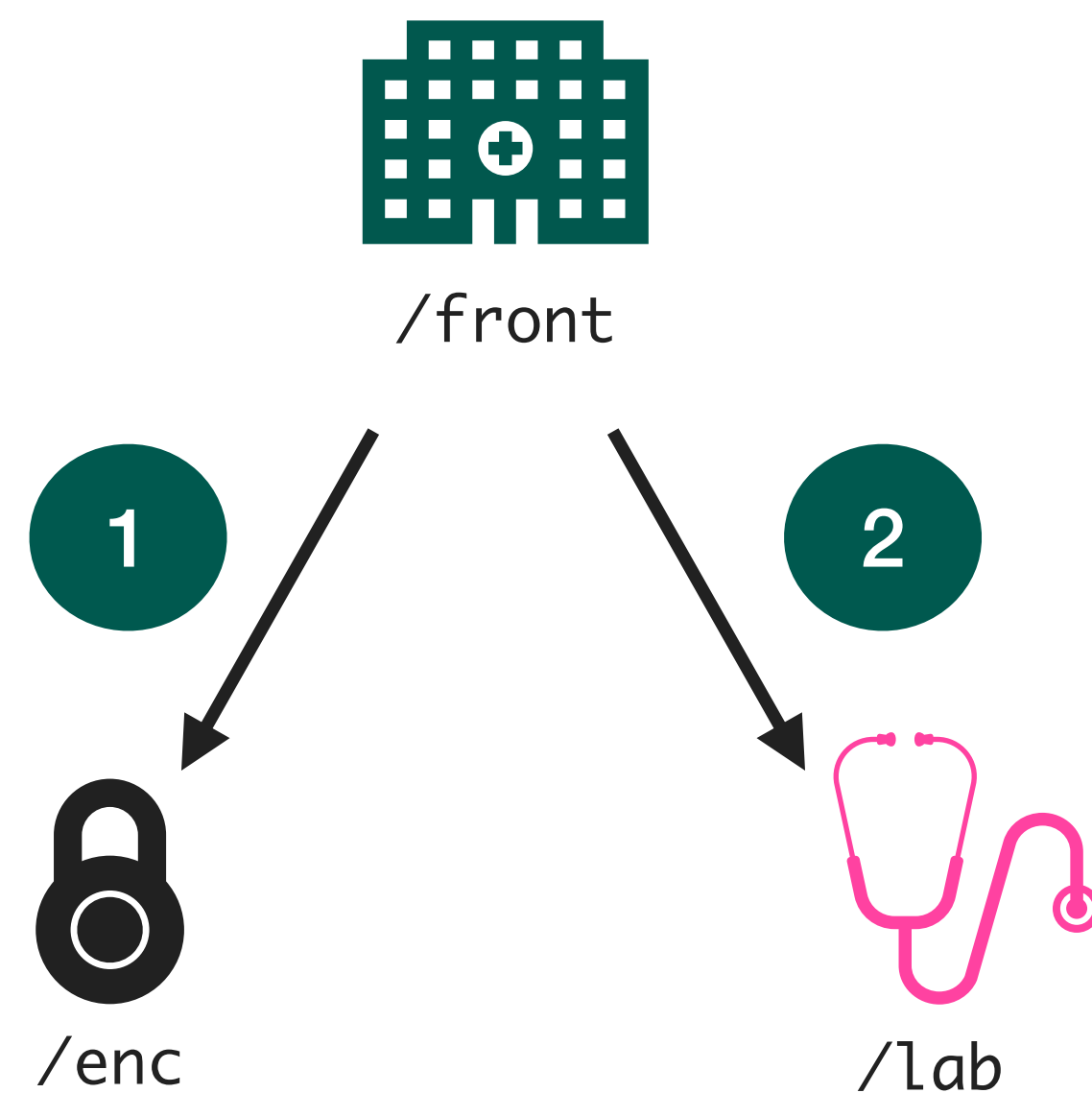
call-Front **call**-Enc **ret**-Enc **call**-Lab **ret**-Lab **ret**-Front

This word appears to be capturing aspects of microservice execution!!!

Let's connect formal notion of words to microservice execution trace!

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

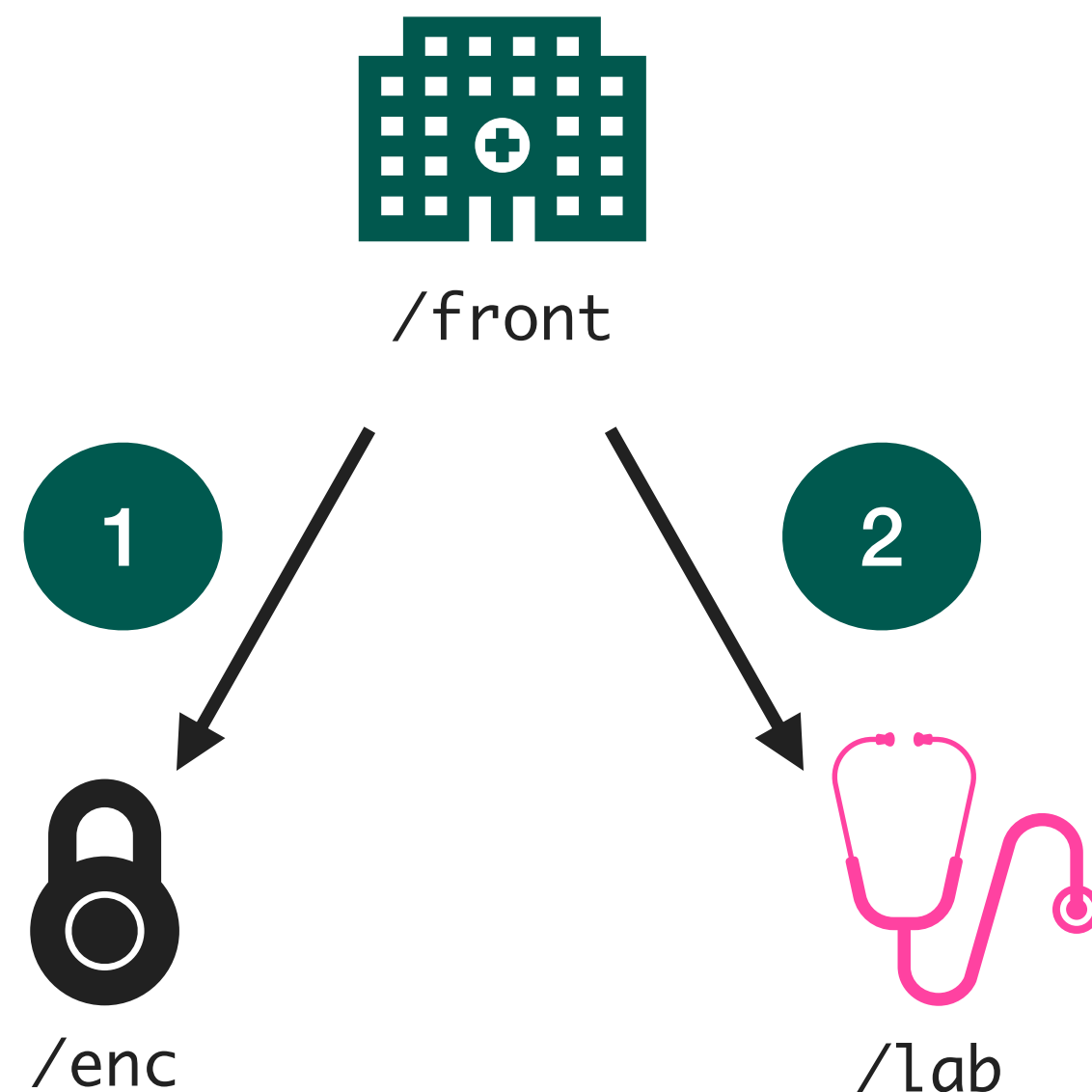
In the execution,
/Front call **happens before** /Enc call ... /Lab **returns before** /Front returns



EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

In the execution,
/Front call happens before /Enc call ... /Lab returns before /Front returns
+

- (1) /Front is the parent
- (2) /Enc is /Front's first child
- (3) /Lab is /Front's second child ...



EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

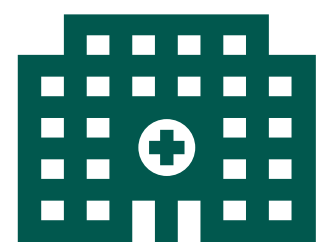
In the execution,
/Front call **happens before** /Enc call ... /Lab **returns before** /Front returns

+

- (1) /Front is the **parent**
- (2) /Enc is /Front's **first child**
- (3) /Lab is /Front's **second child** ...



Child is nested between the parent API



/front

1



/enc

2



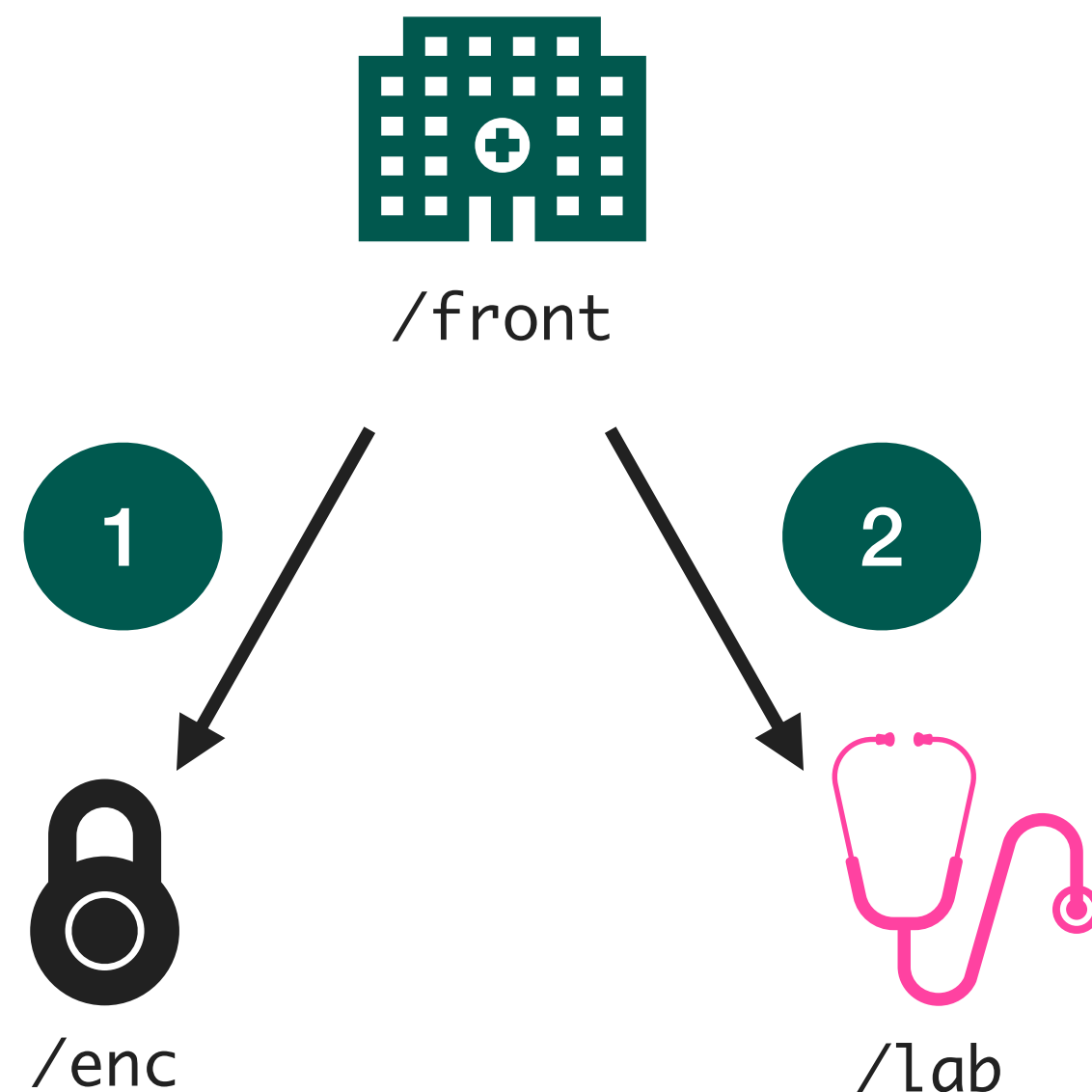
/lab

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

In the execution,

Word of $\mathcal{C} \uplus \mathcal{R}$ can only capture linear structure

- (1) /Front is the **parent**
- (2) /Enc is /Front's **first child**
- (3) /Lab is /Front's **second child** ...



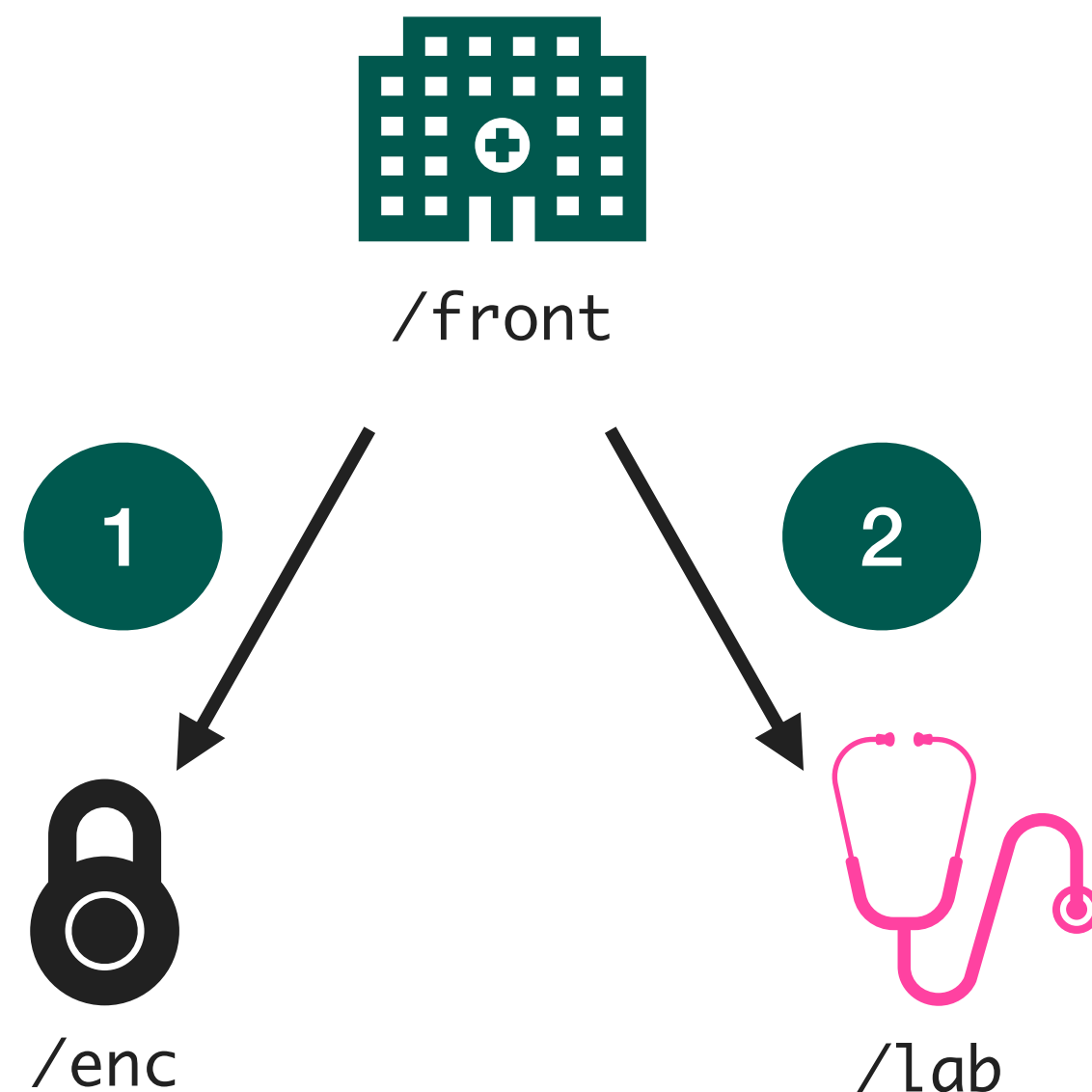
► Linear structure = flat string of APIs

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

In the execution,

Word of $\mathcal{C} \uplus \mathcal{R}$ can only capture linear structure

- (1) /Front is the **parent**
- (2) /Enc is /Front's **first child**
- (3) /Lab is /Front's **second child** ...



► Linear structure = flat string of APIs

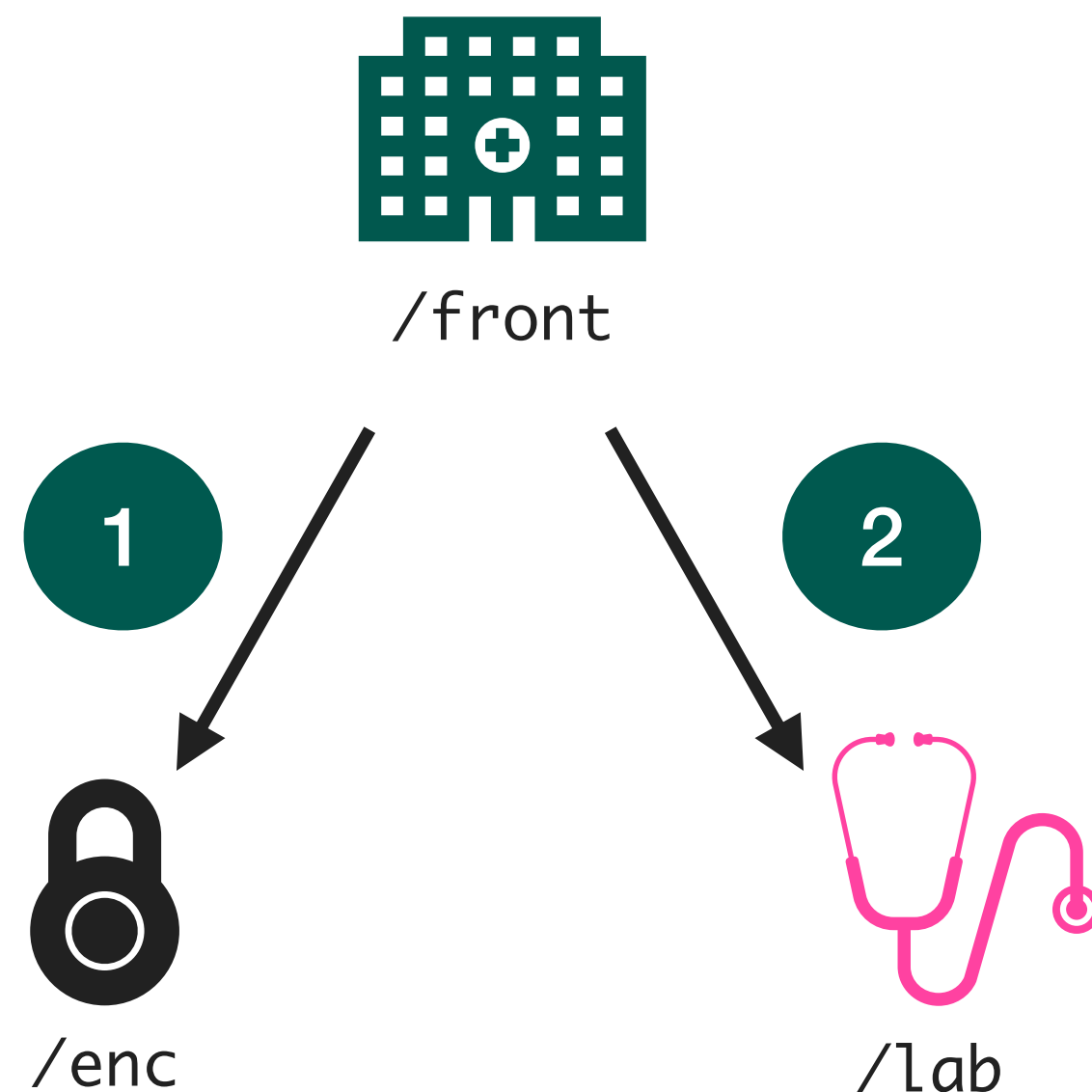
call-Front **call**-Enc **ret**-Enc **call**-Lab **ret**-Lab **ret**-Front

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

In the execution,

Word of $\mathcal{C} \uplus \mathcal{R}$ can only capture linear structure

Carry information about which API call nested which other API calls to capture hierarchical structure



► Linear structure = flat string of APIs

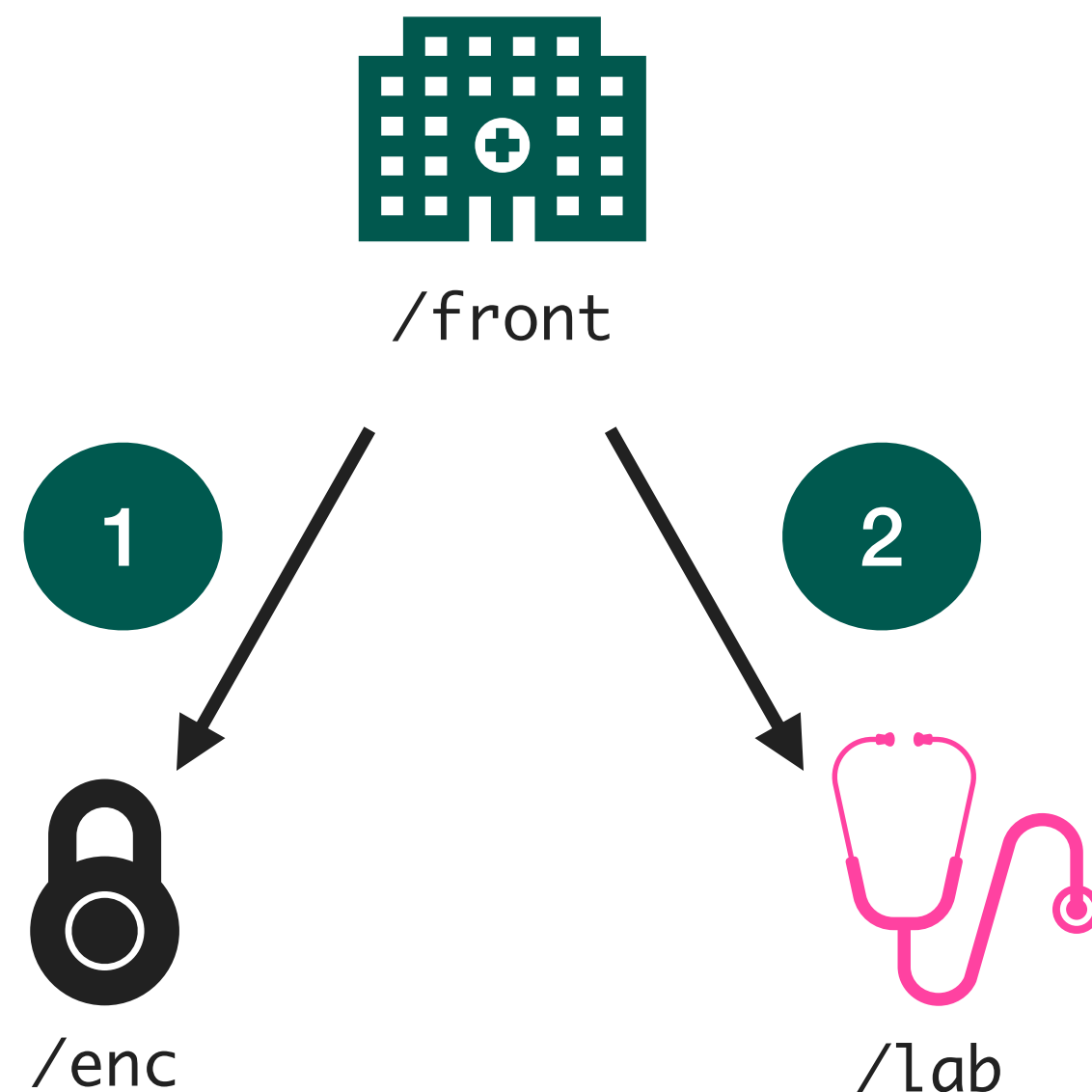
call-Front **call**-Enc **ret**-Enc **call**-Lab **ret**-Lab **ret**-Front

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

In the execution,

Word of $\mathcal{C} \uplus \mathcal{R}$ can only capture linear structure

Carry information about which API call nested which other API calls to capture hierarchical structure



- ▶ Linear structure = flat string of APIs

call-Front **call**-Enc **ret**-Enc **call**-Lab **ret**-Lab **ret**-Front

+

Match Relation

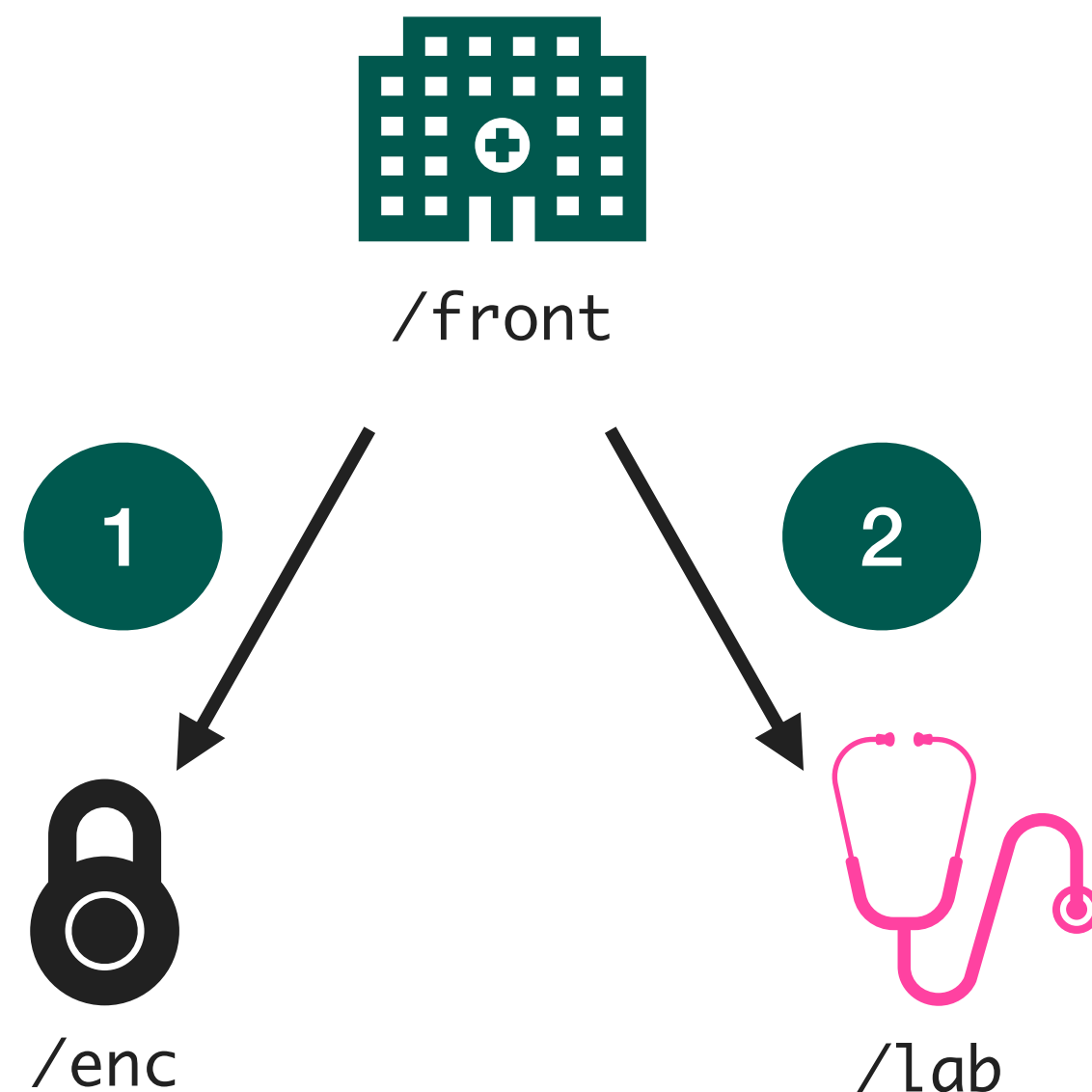
- ▶ Hierarchical structure = children APIs are nested within parent API

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

In the execution,

Word of $\mathcal{C} \uplus \mathcal{R}$ can only capture linear structure

Carry information about which API call nested which other API calls to capture hierarchical structure



- ▶ Linear structure = flat string of APIs

call-Front **call**-Enc **ret**-Enc **call**-Lab **ret**-Lab **ret**-Front

+

Match Relation

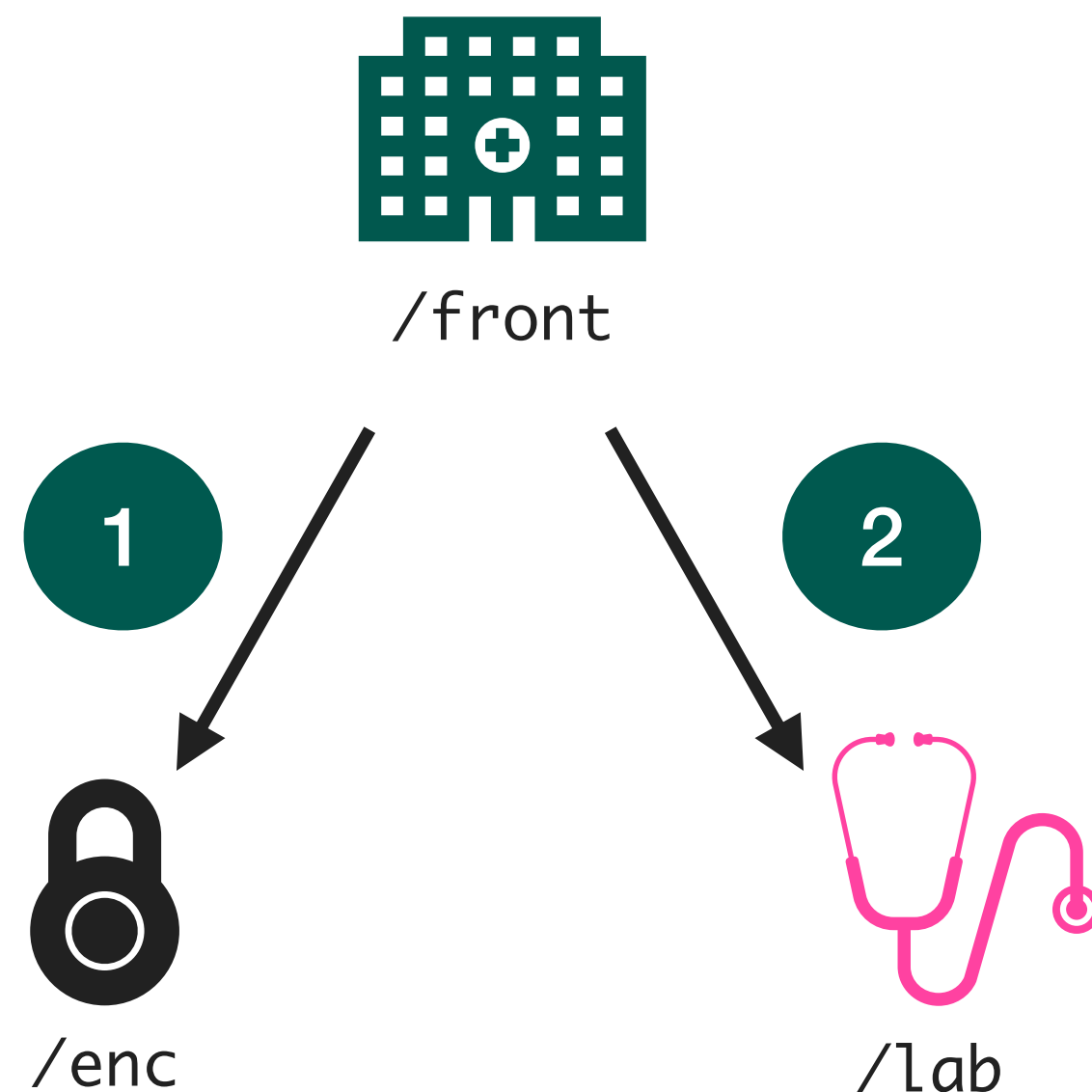
- ▶ Hierarchical structure = children APIs are nested within parent API

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

In the execution,

Word of $\mathcal{C} \uplus \mathcal{R}$ can only capture linear structure

Carry information about which API call nested which other API calls to capture hierarchical structure



- ▶ Linear structure = flat string of APIs

call-Front **call**-Enc **ret**-Enc **call**-Lab **ret**-Lab **ret**-Front

+

Match Relation

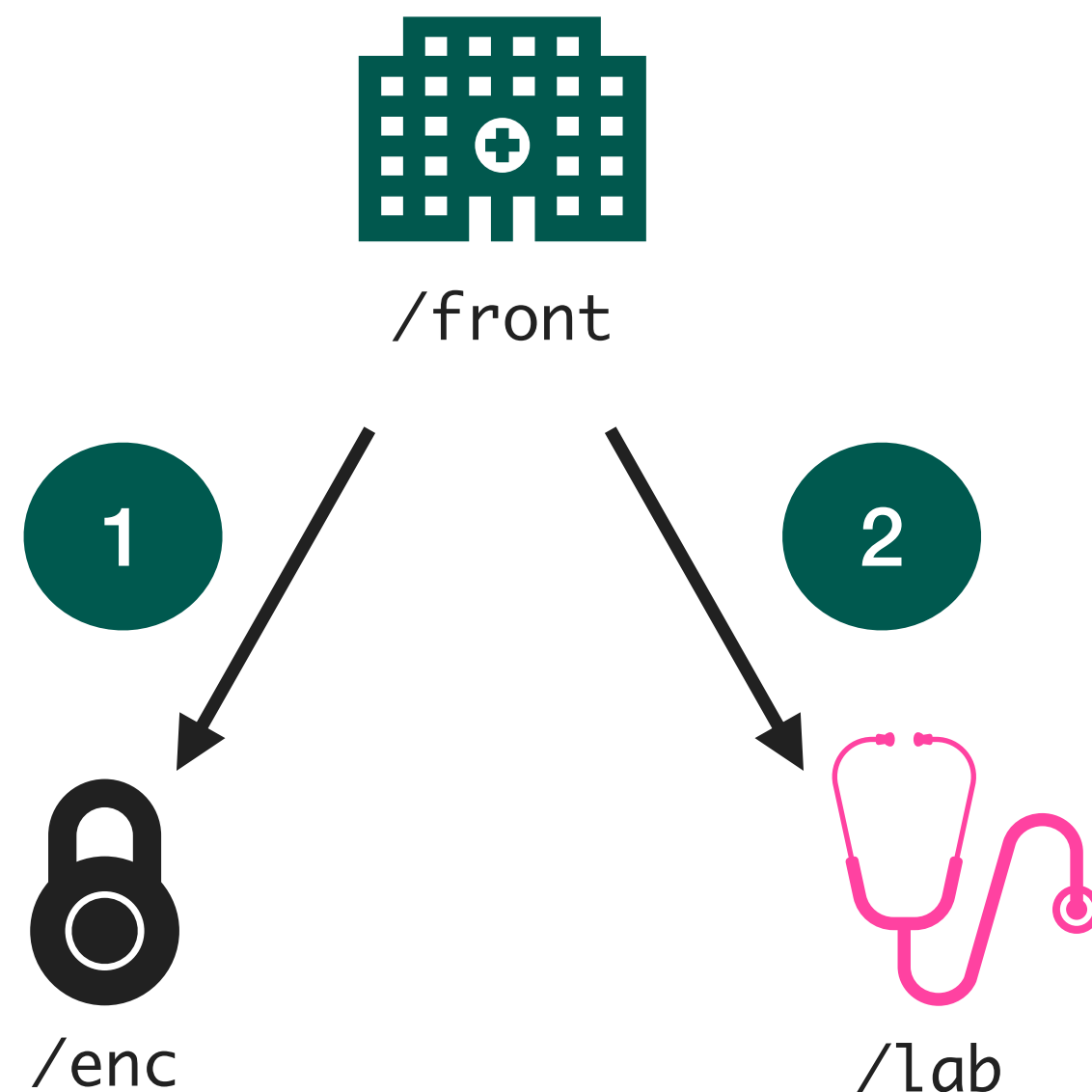
- ▶ Hierarchical structure = children APIs are nested within parent API

EXECUTION TRACE HAS LINEAR + HIERARCHICAL STRUCTURE

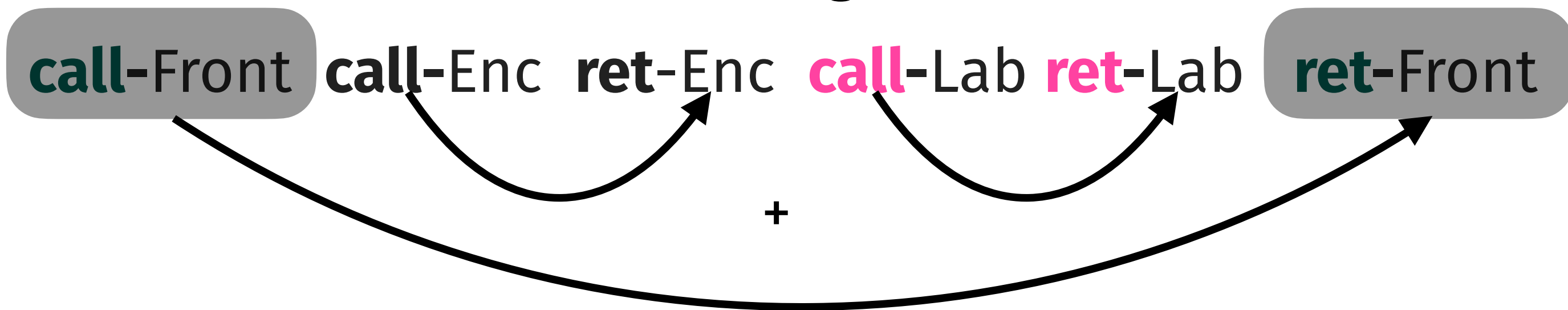
In the execution,

Word of $\mathcal{C} \uplus \mathcal{R}$ can only capture linear structure

Carry information about which API call nested which other API calls to capture hierarchical structure



► Linear structure = flat string of APIs



Match Relation

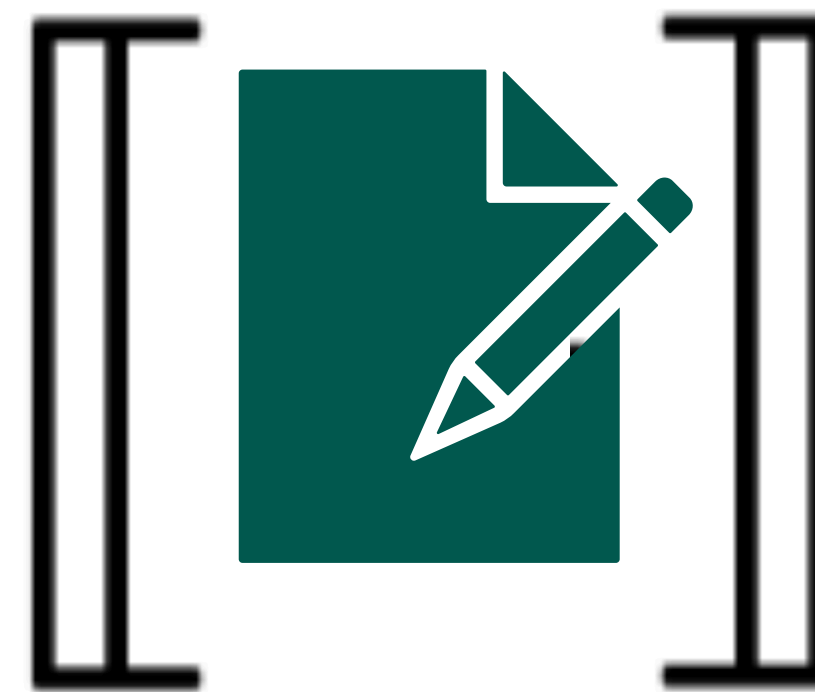
► Hierarchical structure = children APIs are nested within parent API

SAFETREE POLICIES = SET OF NESTED WORDS

Nested Word over $\mathcal{C} \uplus \mathcal{R}$ is a pair of (flat word, match relation)

We define:

Nested-Words

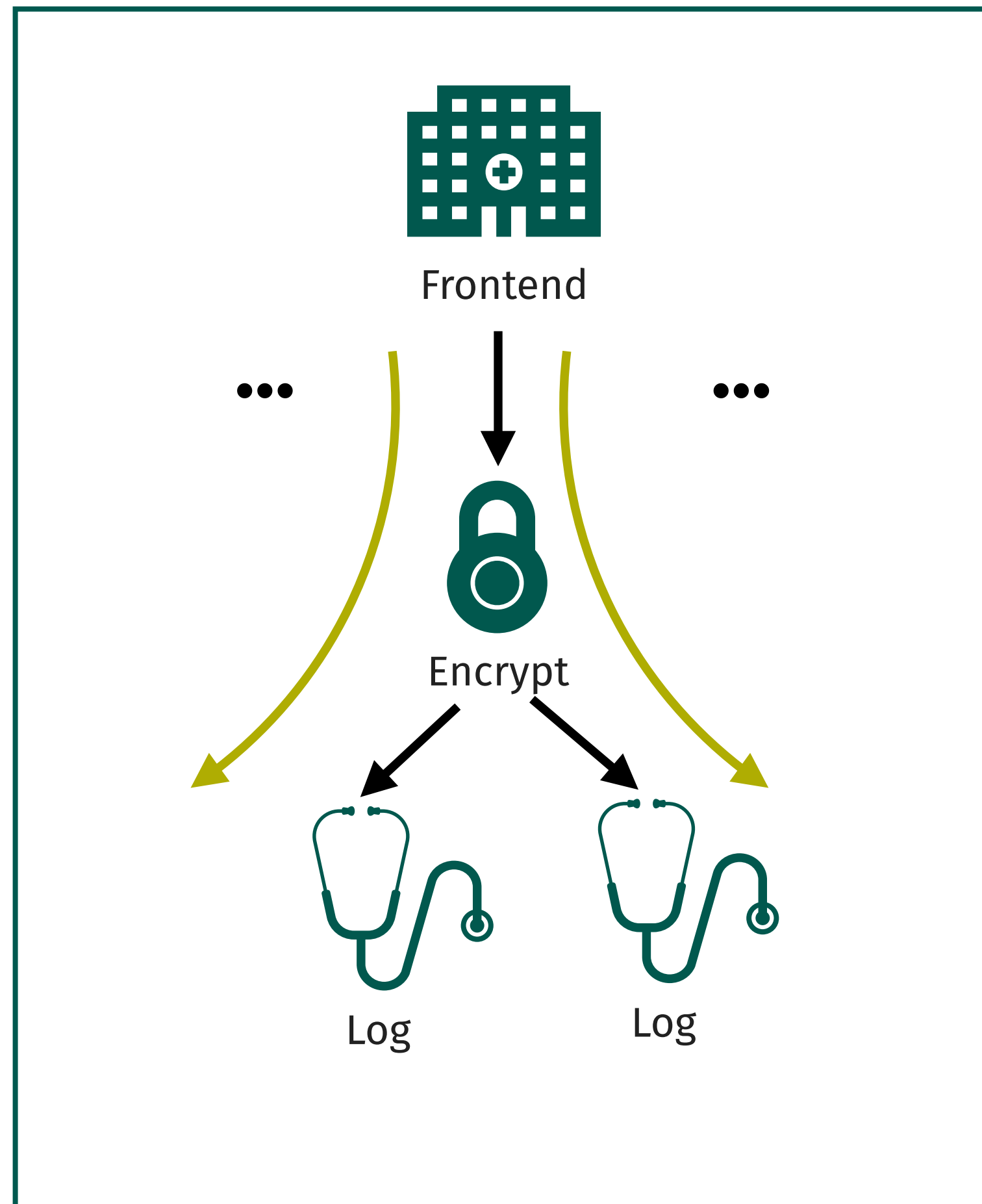


SafeTree Policy

SafeTree Language: Syntax



SAFETREE LANGUAGE AT A GLANCE

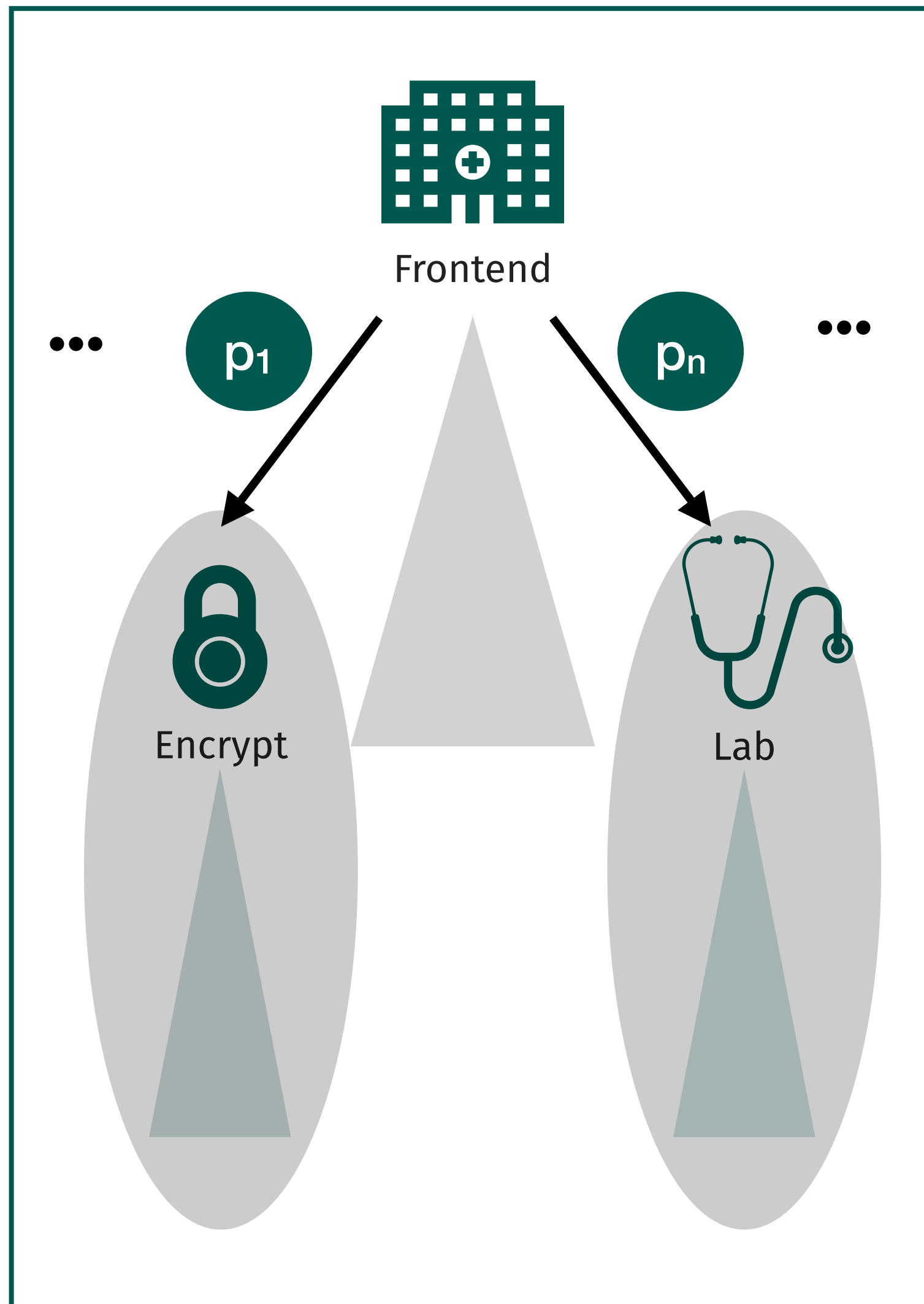


► `match root` $\xRightarrow{\forall\text{-path}}$ `regex`

Match regex on all outgoing paths from root

On all paths from Frontend “Log” should be called

SAFETREE LANGUAGE AT A GLANCE

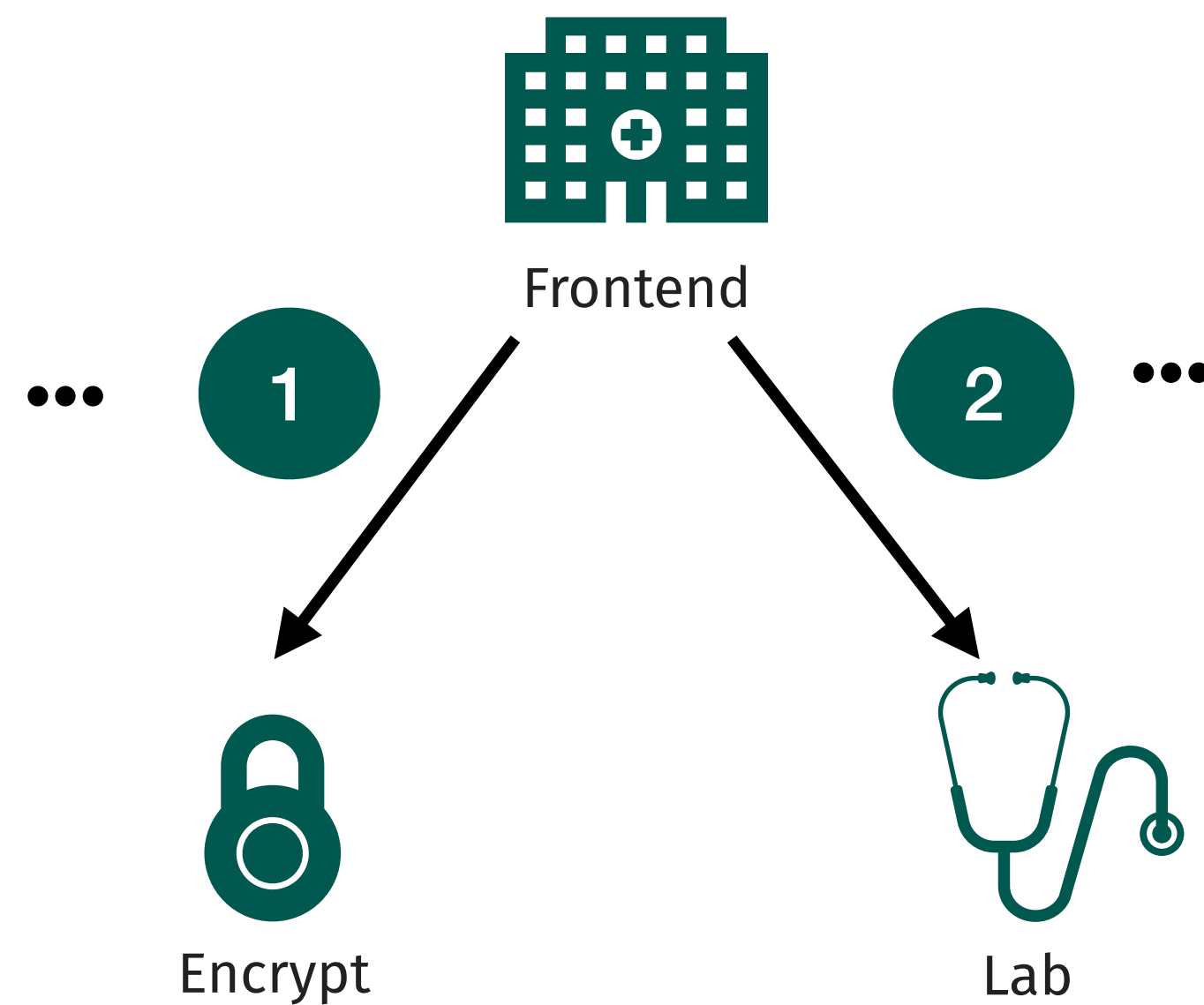


► `match root  $\xRightarrow{\exists\text{-child}}$  p_1 \text{ then } \dots \text{ then } p_n`

“root” should have at least “n” subtrees.

Subtrees should satisfy `p1 then ... then pn` in that order

REVISITING OUR ENCRYPTION POLICY



Let's say Encrypt and Lab
should be leaf nodes

match Frontend $\xRightarrow{\exists\text{-child}}$ p_1 **then** p_2

p_1 **match** Encrypt $\xRightarrow{\forall\text{-path}}$ ε

p_2 **match** Lab $\xRightarrow{\forall\text{-path}}$ ε

ε : Regular expression for no path

SAFETREE LANGUAGE AT A GLANCE

Hierarchical

- ▶ `match regexp` $\xRightarrow{\exists\text{-child}}$...
- ▶ `match regexp` $\xRightarrow{\forall\text{-child}}$...
- ▶ `match regexp` $\xRightarrow{\forall\text{-path}}$...

Linear

- ▶ `call-sequence regexp`

Policies are compositional

Enforcement: Visibly Pushdown Automaton



Policy



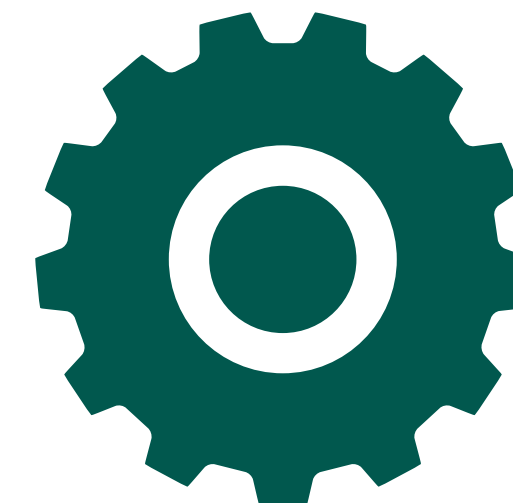
Runtime Monitor

COMPILING POLICY TO VISIBLY PUSHDOWN AUTOMATA

Set of well-matched nested words = Language of VPA



Policy



VPA

COMPILING POLICY TO VISIBLY PUSHDOWN AUTOMATA

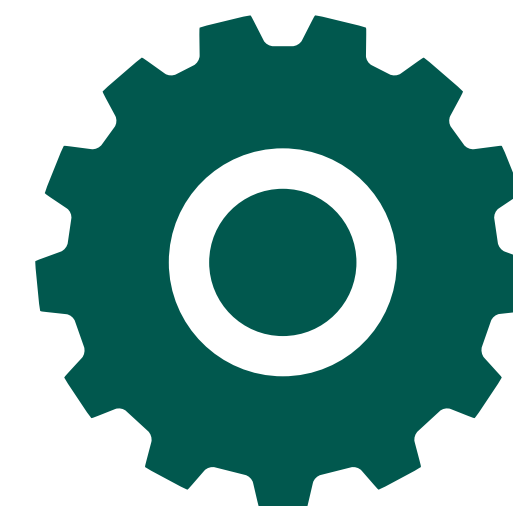
Set of well-matched nested words = Language of VPA

We propose a SafeTree compiler to generate the VPA for the policy



Policy

Compiler
→



VPA

POLICY IS SOUNDLY COMPILED TO THE VPA

Prove soundness theorem of the **VPA** wrt the **nested word interpretation**:

- Any nested word allowed by the policy is also accepted by the VPA
- Any nested word accepted by the VPA is also allowed by the policy

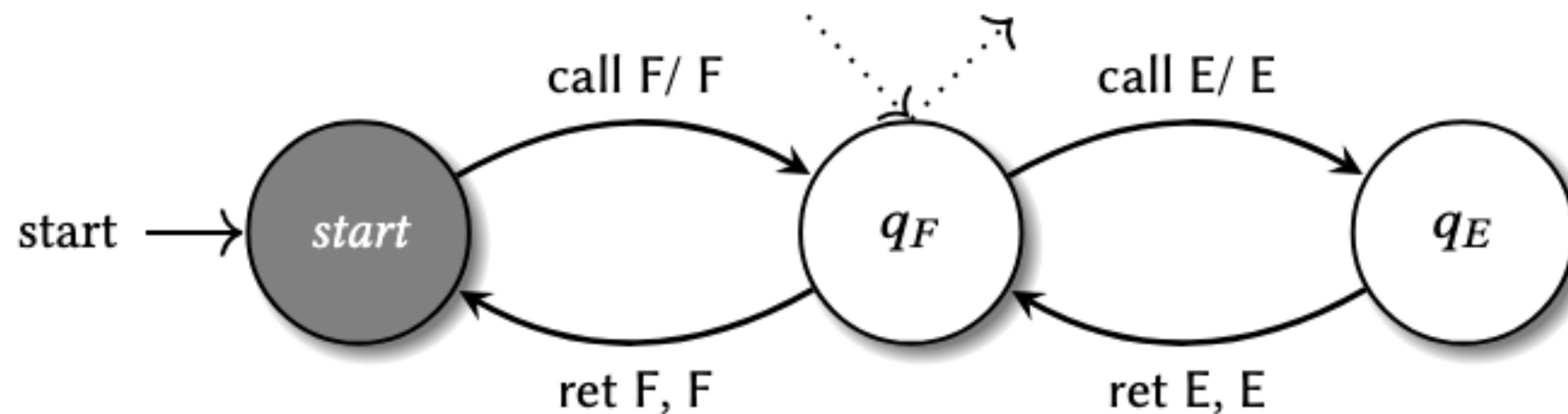
$$\textit{Nested-Words} \left[\begin{array}{c} \text{Policy} \end{array} \right] \equiv \textit{Language of compiled VPA}$$

Bonus slides for understanding VPA internals

BONUS: VISIBLY PUSHDOWN AUTOMATA TO RECOGNIZE VALID

Definition (Finite state automaton + stack)

A set of states = $\{start, q_F, q_E\}$ Initial state = $start$ Final state = $\{start\}$



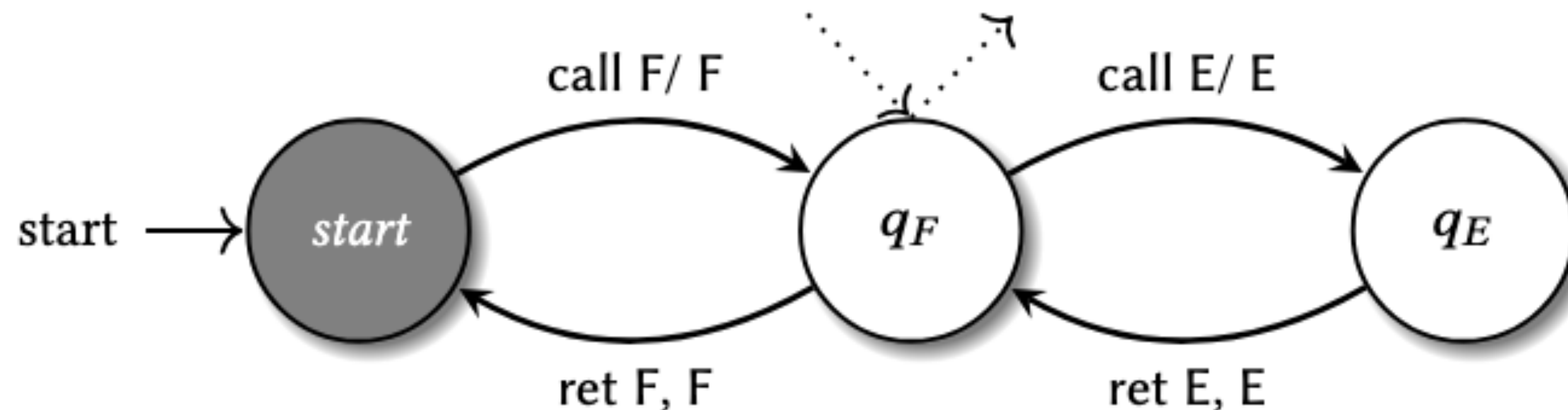
BONUS: VISIBLY PUSHDOWN AUTOMATA TO RECOGNIZE VALID

Definition (Finite state automaton + stack)

A set of states = $\{start, q_F, q_E\}$ Initial state = start Final state = $\{start\}$

Two transition functions:

(a) $\delta_{call}(\text{state}, \text{transition symbol}) = (\text{next state}, \text{symbol to push on stack})$



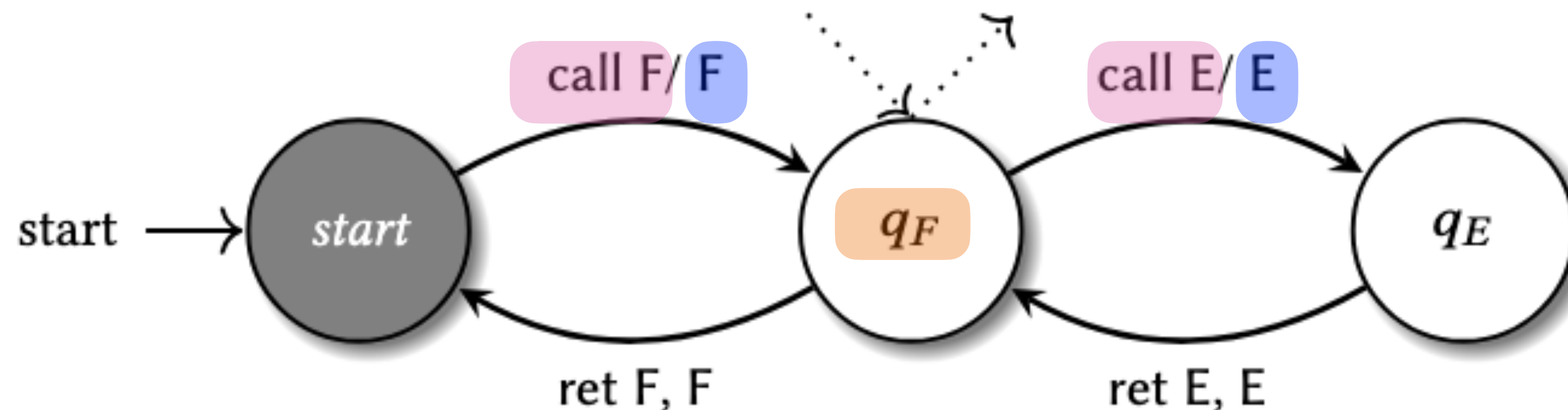
BONUS: VISIBLY PUSHDOWN AUTOMATA TO RECOGNIZE VALID

Definition (Finite state automaton + stack)

A set of states = $\{start, q_F, q_E\}$ Initial state = start Final state = $\{start\}$

Two transition functions:

(a) $\delta_{call}(\text{state}, \text{transition symbol}) = (\text{next state}, \text{symbol to push on stack})$



BONUS: VISIBLY PUSHDOWN AUTOMATA TO RECOGNIZE VALID

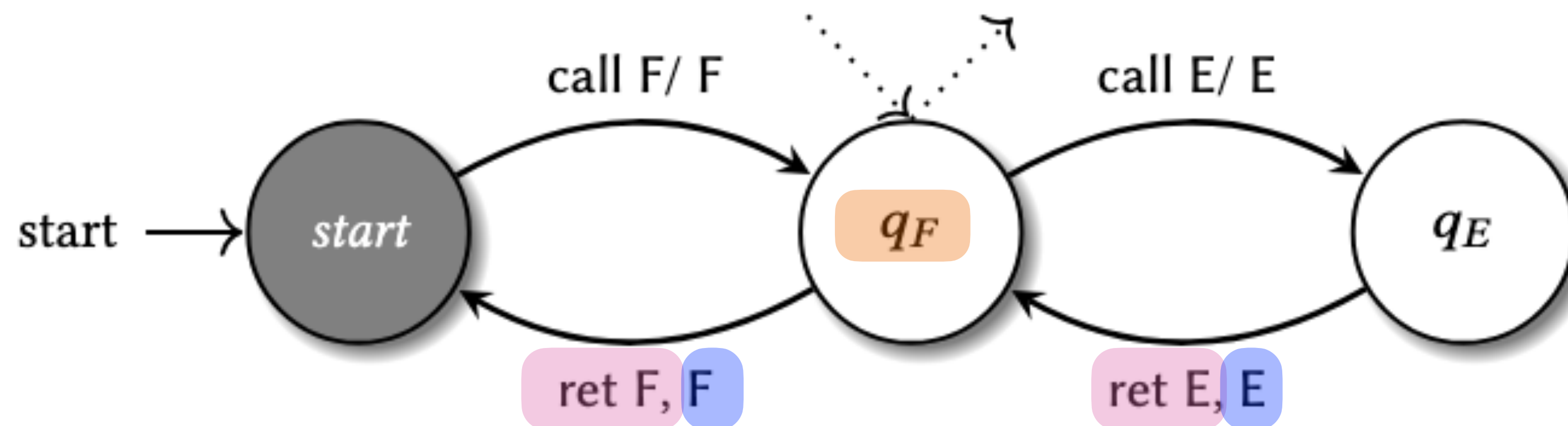
Definition (Finite state automaton + stack)

A set of states = $\{start, q_F, q_E\}$ Initial state = start Final state = $\{start\}$

Two transition functions:

(a) $\delta_{call}(\text{state}, \text{transition symbol}) = (\text{next state}, \text{symbol to push on stack})$

(b) $\delta_{ret}(\text{state}, \text{transition symbol}, \text{top of stack symbol}) = \text{next state}$



BONUS: VISIBLY PUSHDOWN AUTOMATA TO RECOGNIZE VALID

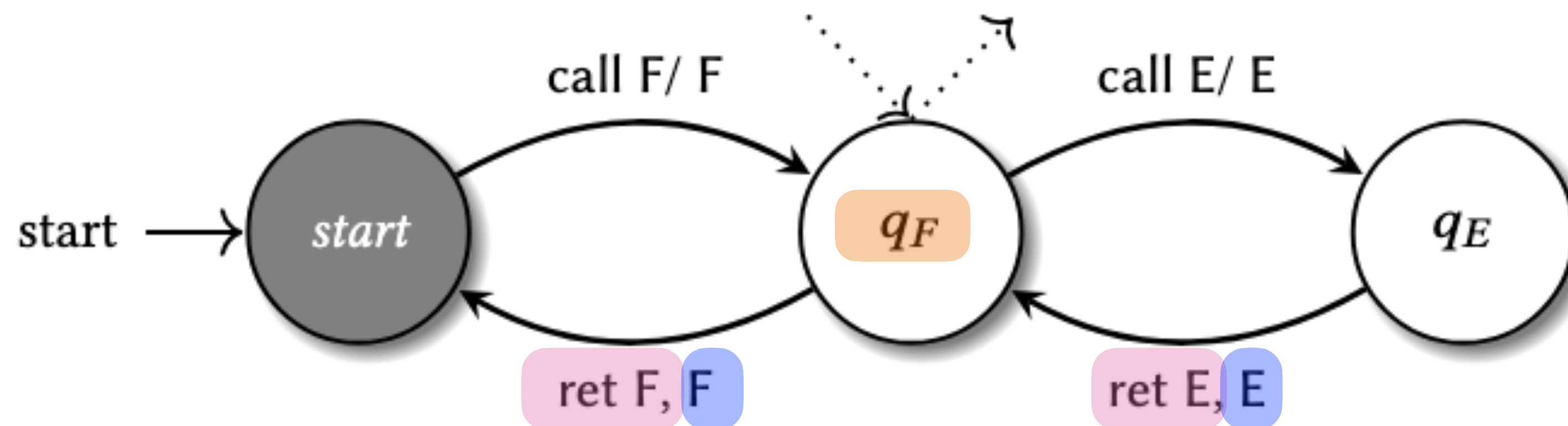
Definition (Finite state automaton + stack)

A set of states = $\{start, q_F, q_E\}$ Initial state = start Final state = $\{start\}$

Two transition functions:

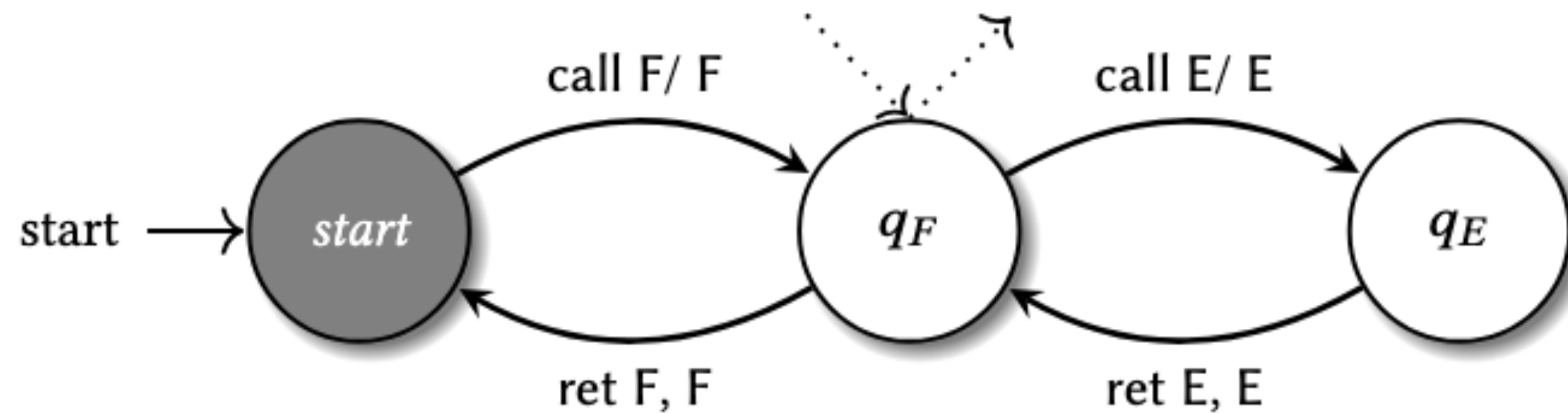
(a) $\delta_{call}(\text{state}, \text{transition symbol}) = (\text{next state}, \text{symbol to push on stack})$

(b) $\delta_{ret}(\text{state}, \text{transition symbol}, \text{top of stack symbol}) = \text{next state}$



BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”



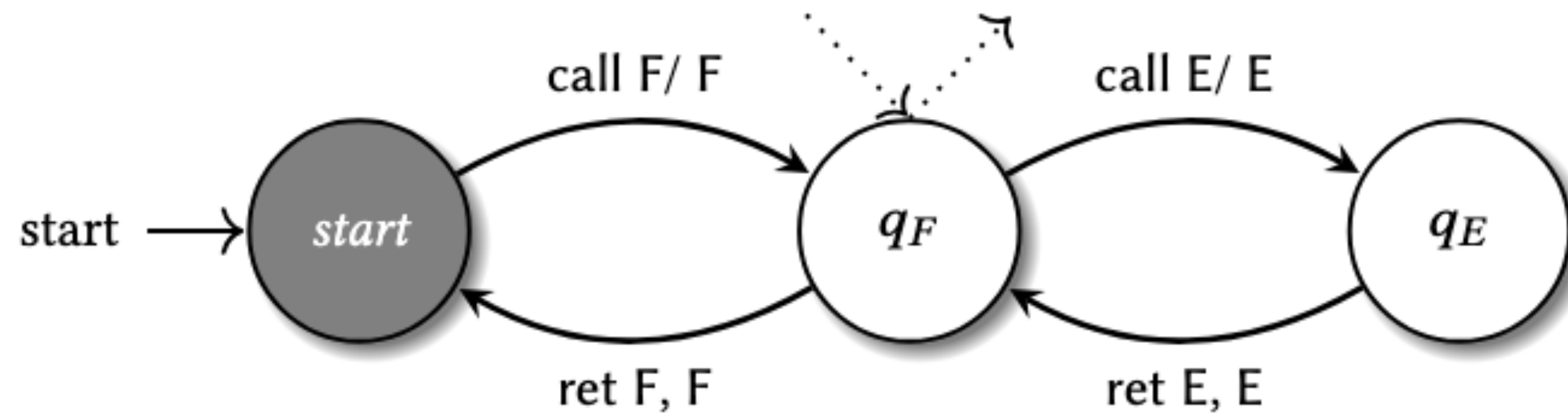
State

Stack

Configuration

BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”



Symbol read

callF

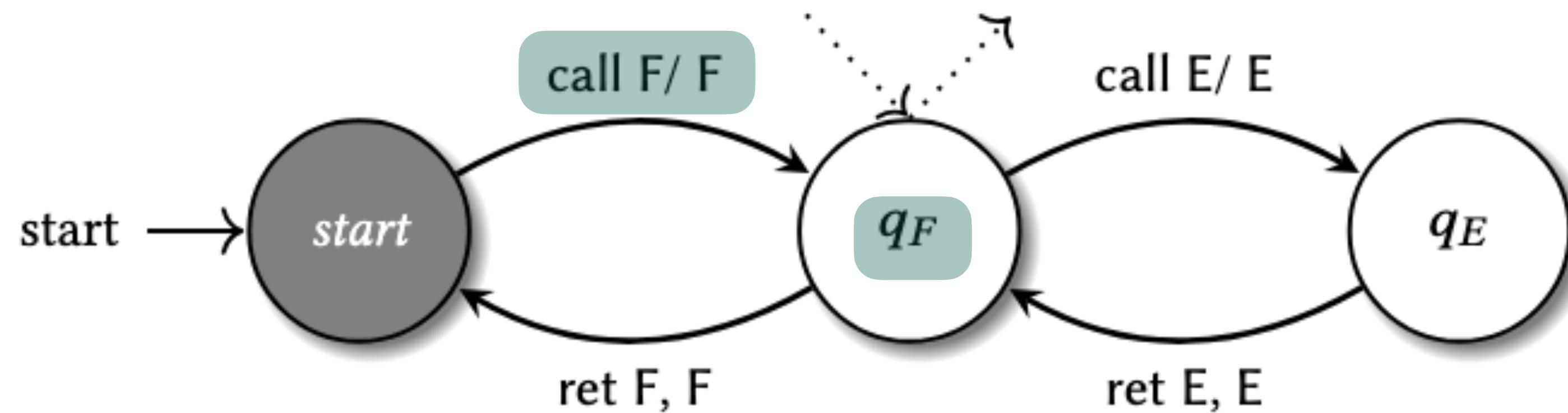
State

Stack

Configuration

BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”



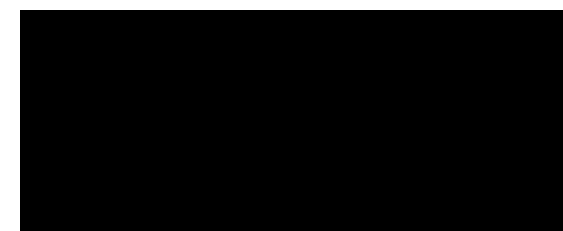
Symbol read

callF

State

start

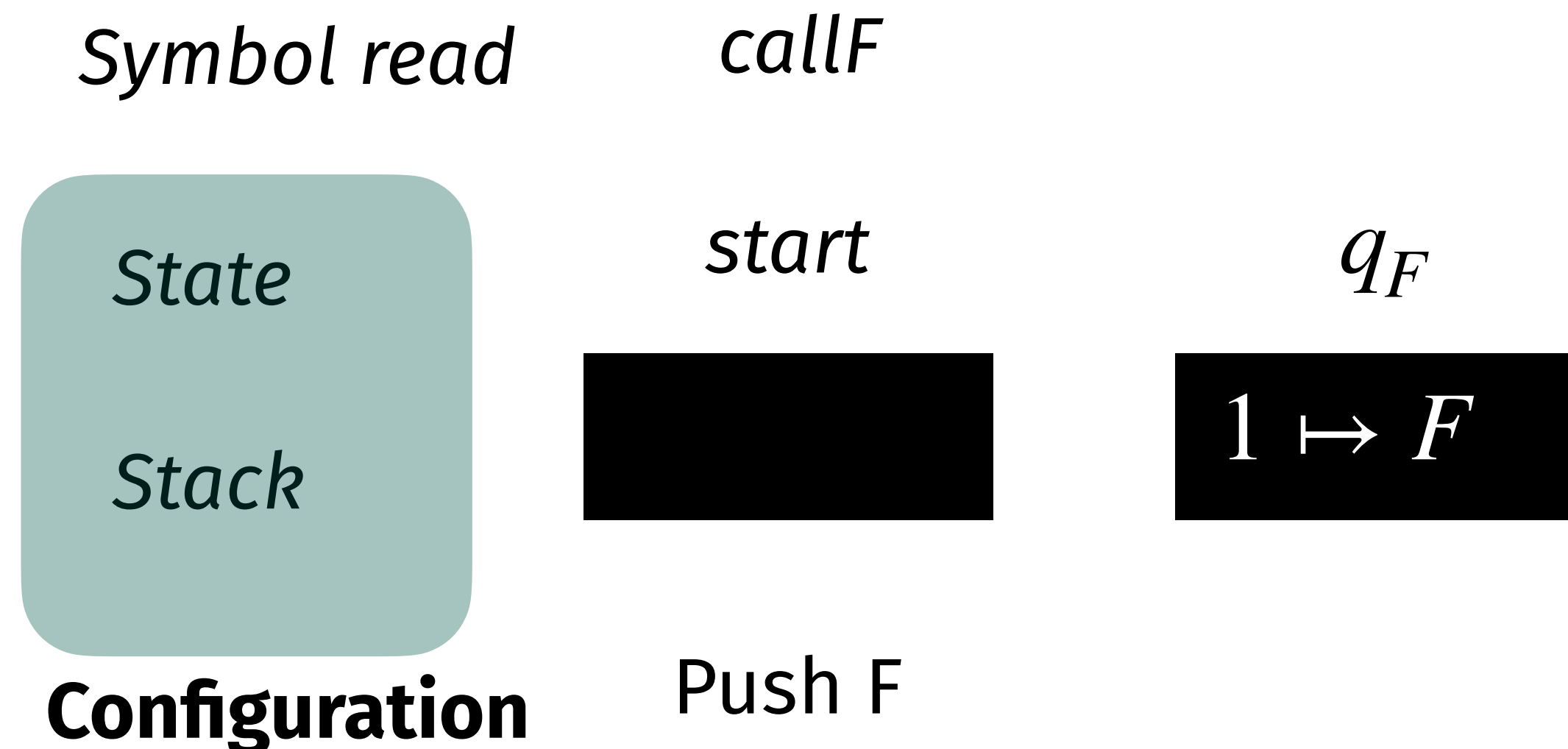
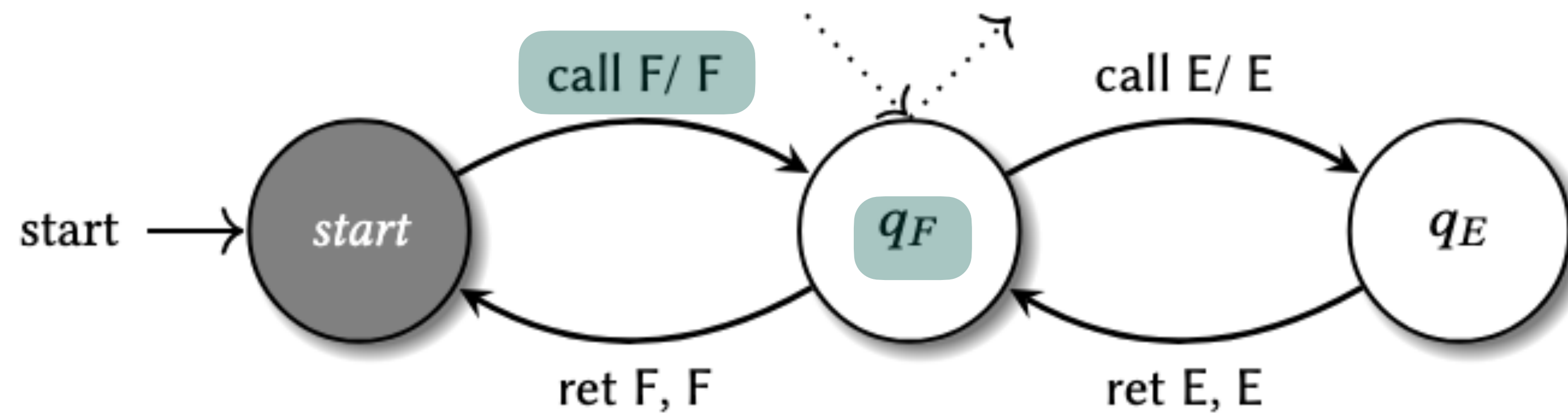
Stack



Configuration

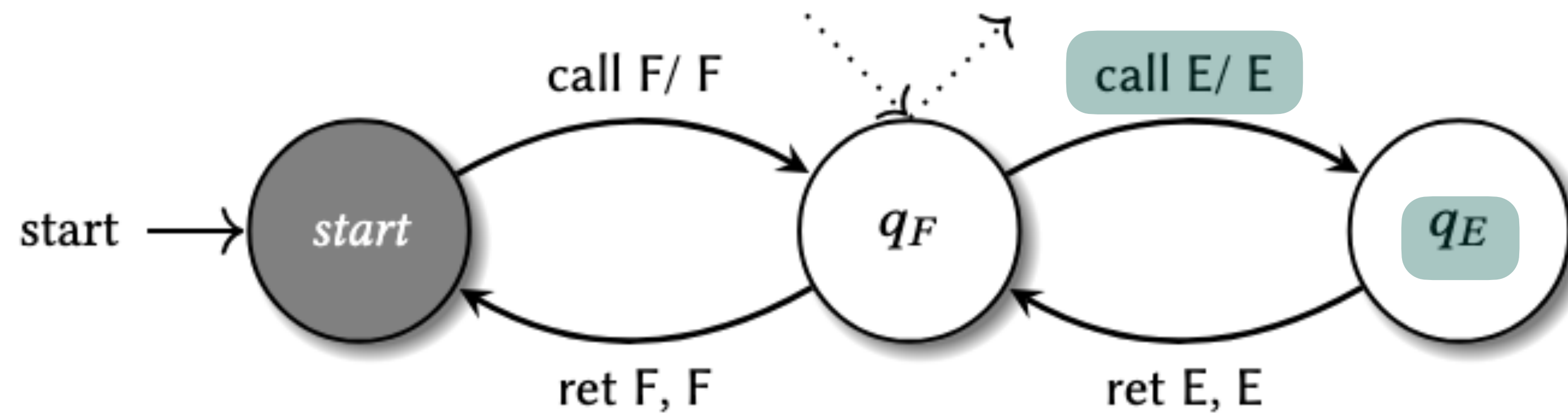
BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”



BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”



Symbol read

callF

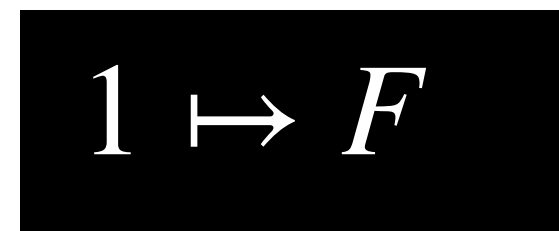
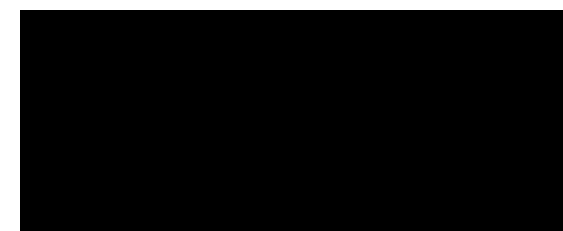
callE

State

start

q_F

Stack

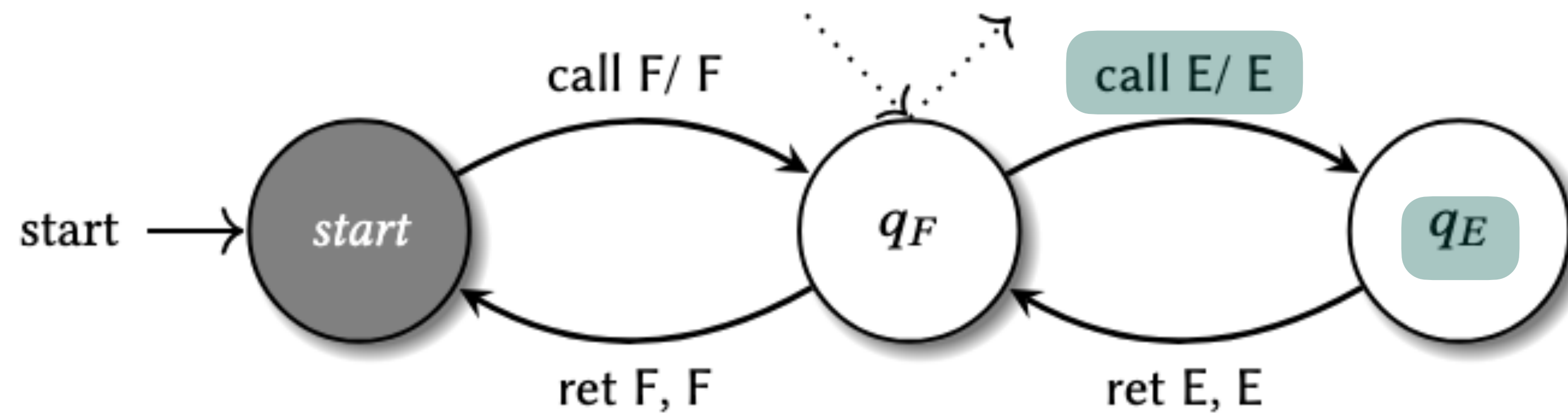


Configuration

Push F

BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”



Symbol read

callF

callE

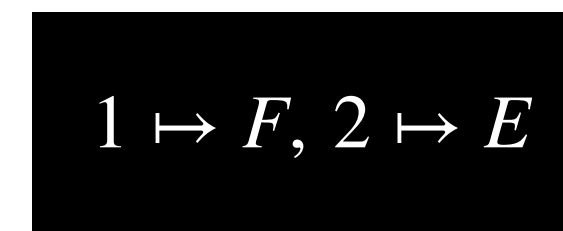
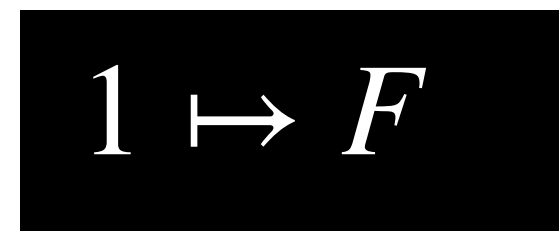
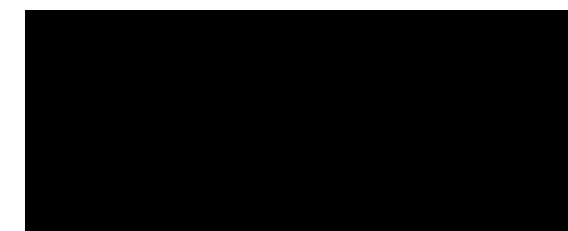
State

start

q_F

q_E

Stack



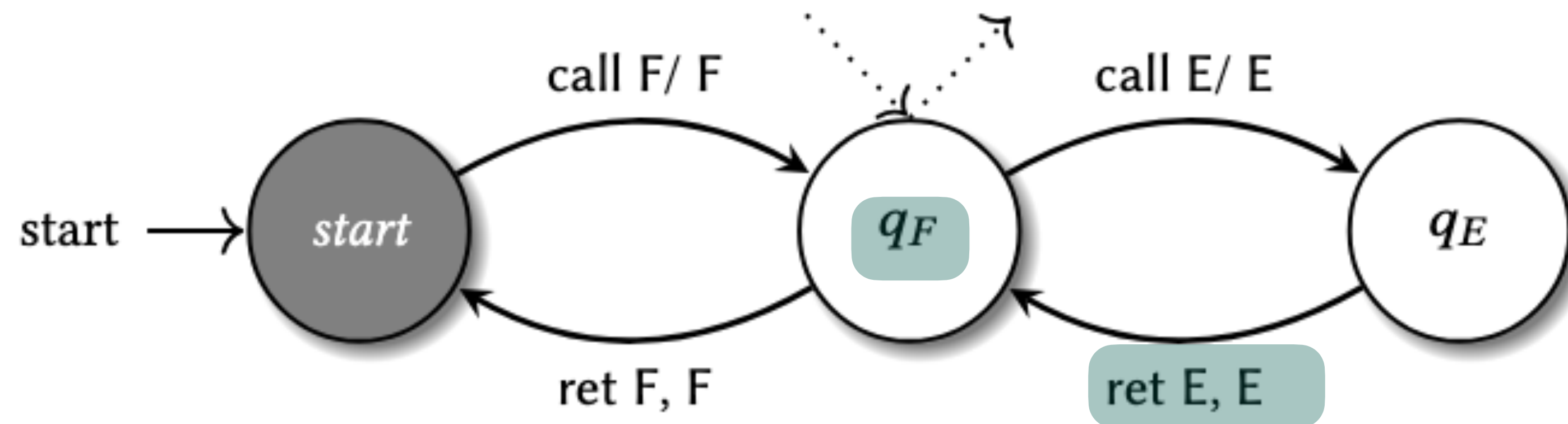
Configuration

Push F

Push E

BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”



Symbol read

callF

callE

retE

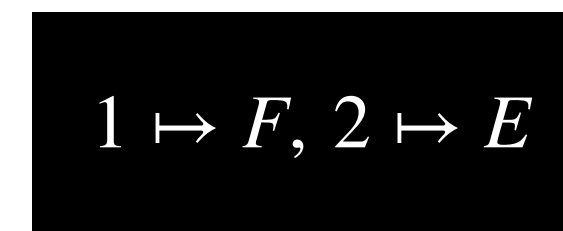
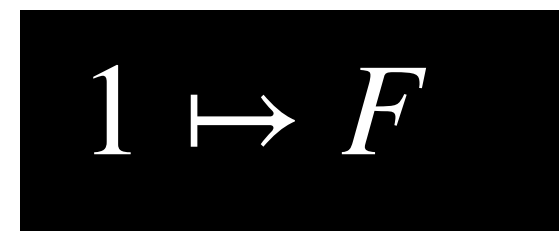
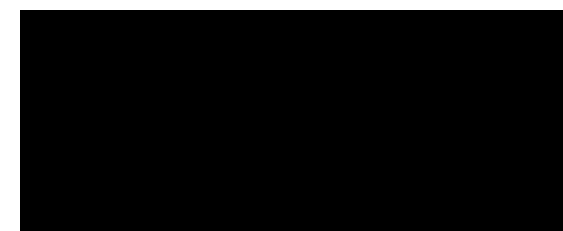
State

start

q_F

q_E

Stack



Configuration

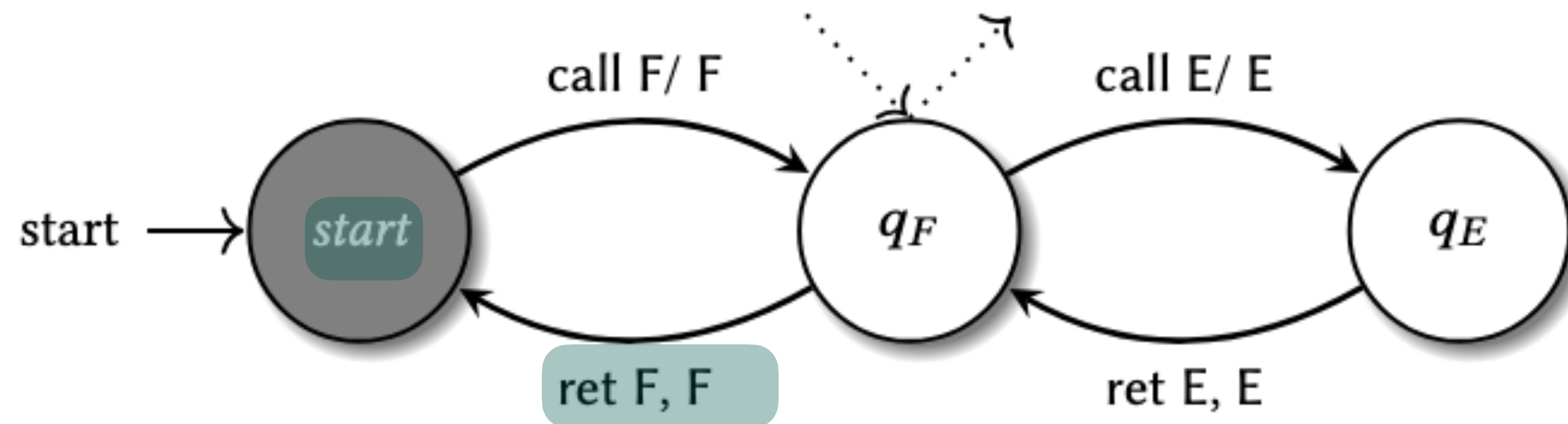
Push F

Push E

Is stack top = E?
Yes; Pop

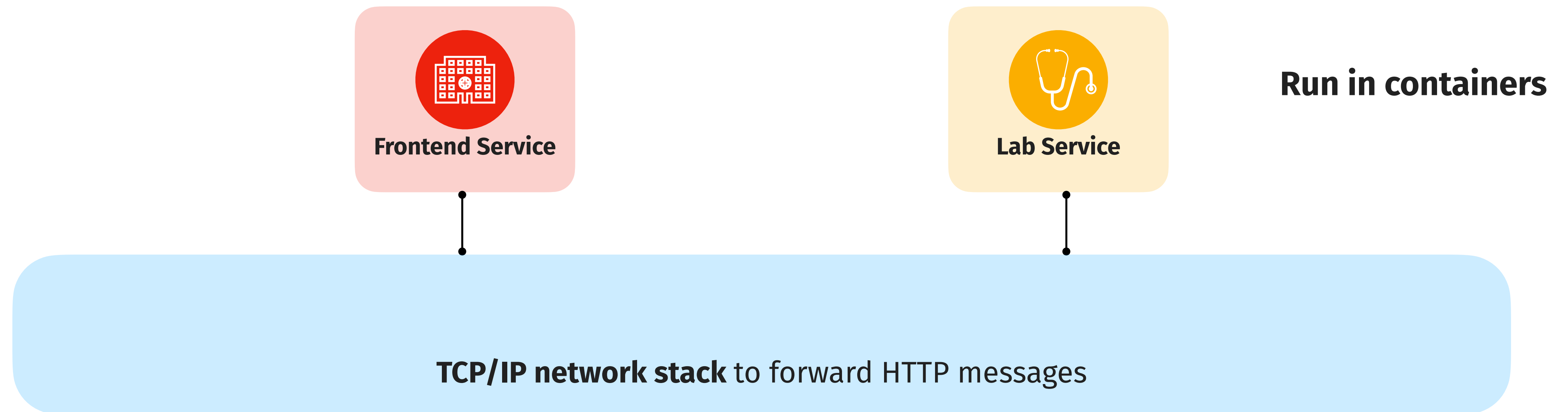
BONUS: SIMULATE THE VPA

Consider trace: callF callE retE retF for the policy “F can only call E as its child”

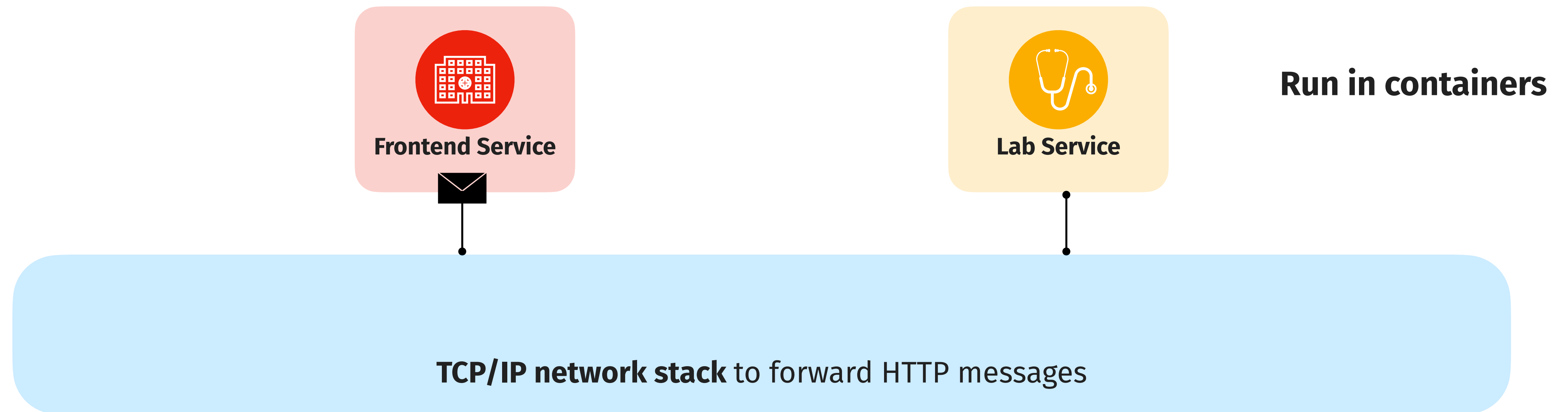


Symbol read	callF	callE	retE	retF	
State	start	q_F	q_E	q_F	start
Stack		$1 \mapsto F$	$1 \mapsto F, 2 \mapsto E$	$1 \mapsto F$	
Configuration	Push F	Push E	Is stack top = E? Yes; Pop	Is stack top = F? Yes; Pop	

SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK

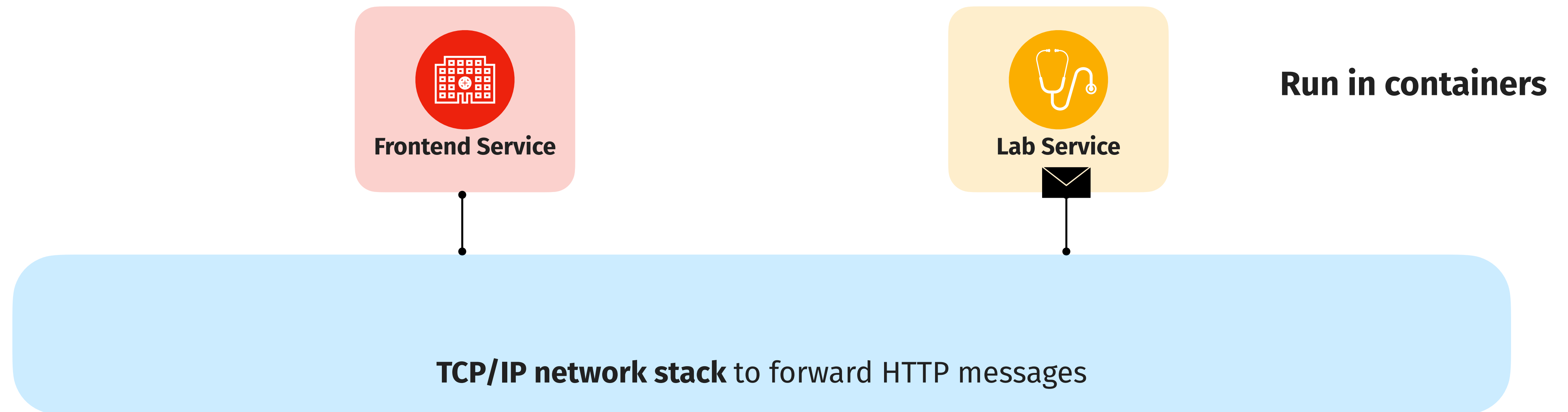


SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK



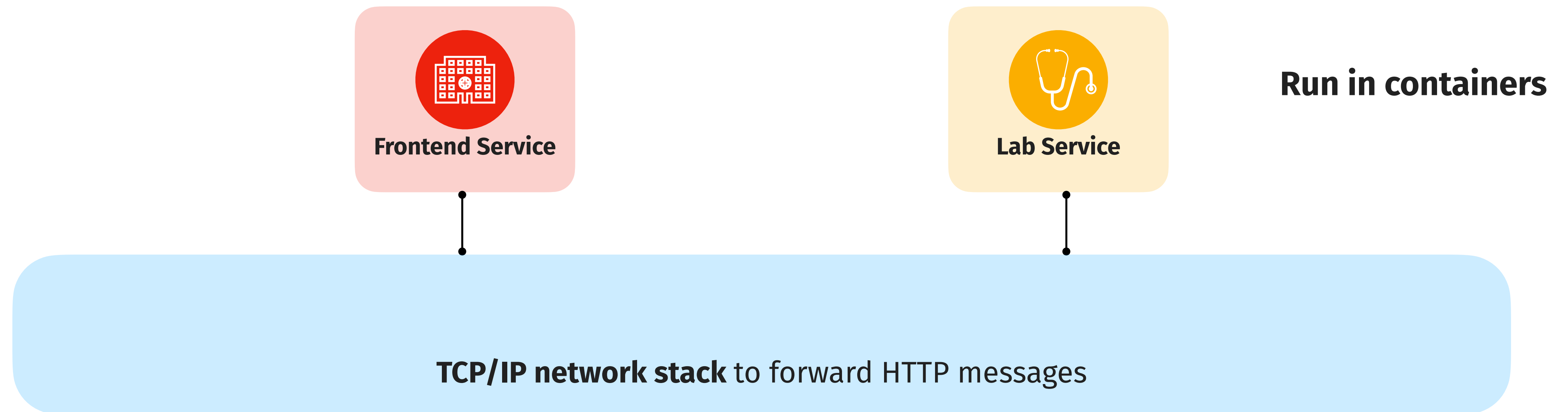
Consider an example HTTP message flow from Frontend to Lab service

SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK

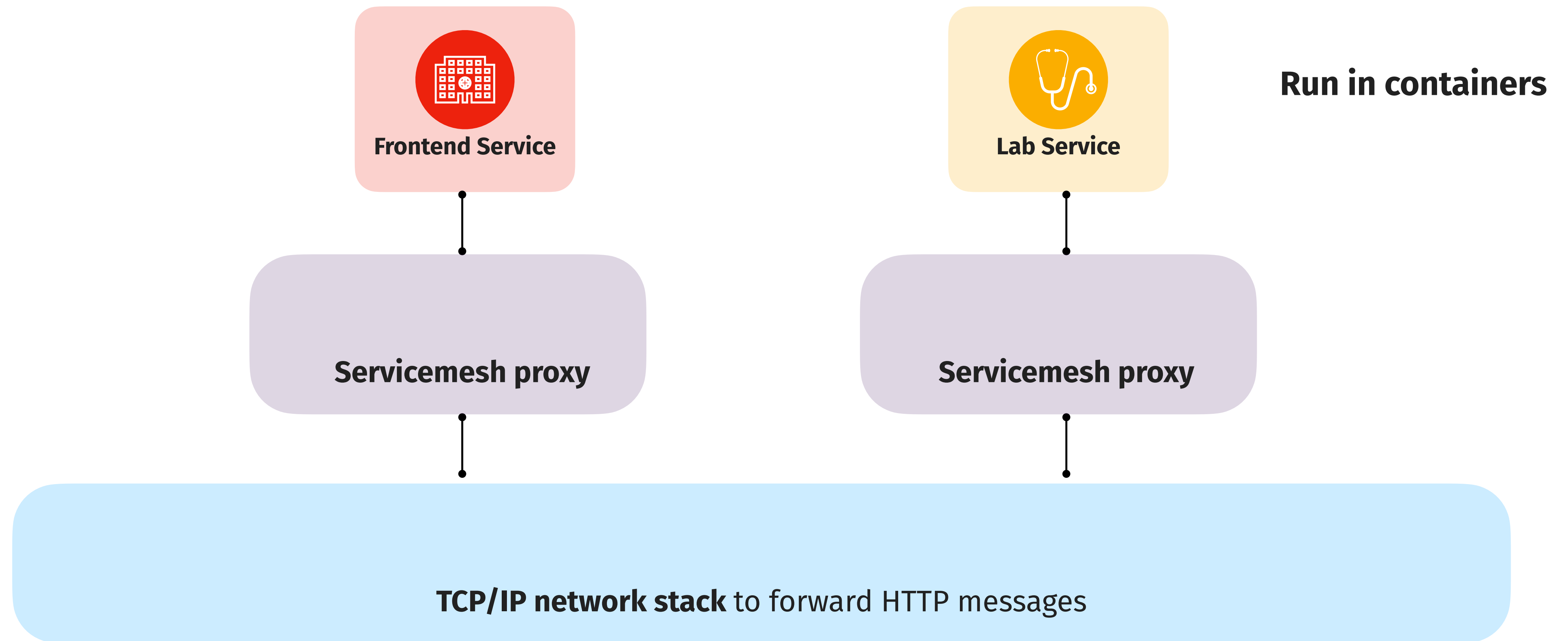


Consider an example HTTP message flow from Frontend to Lab service

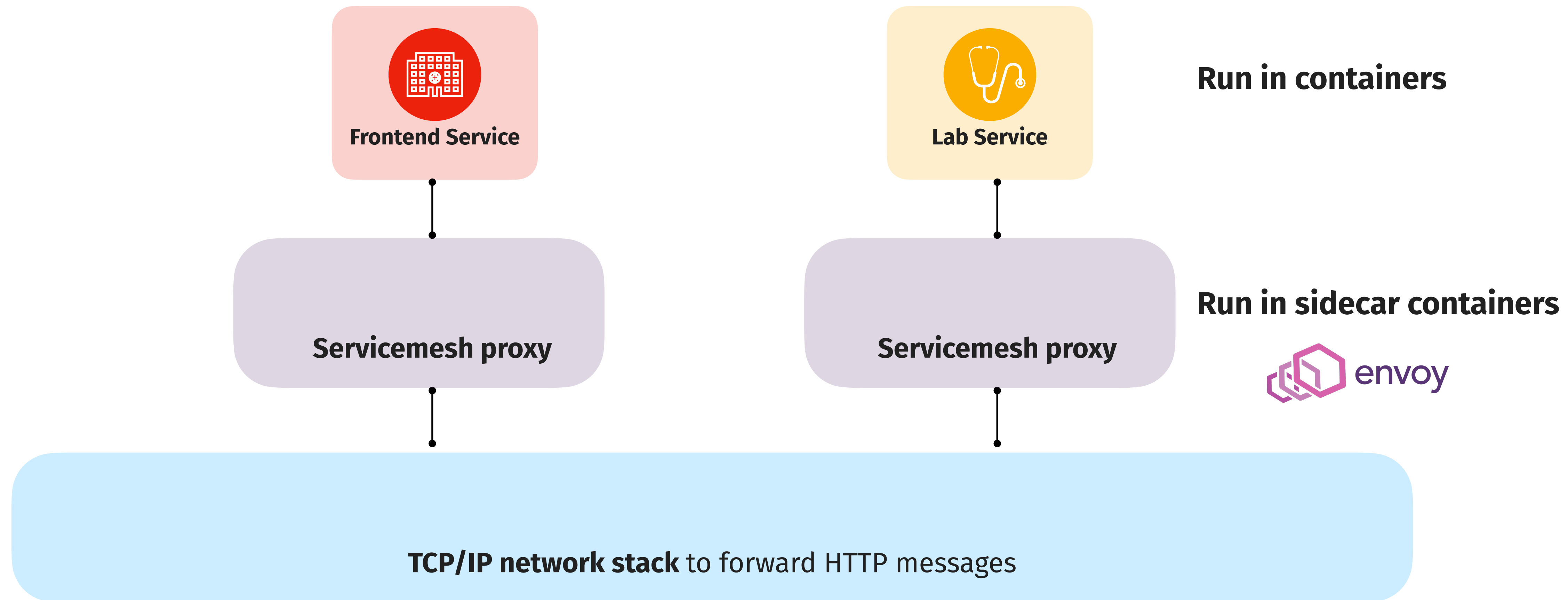
SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK



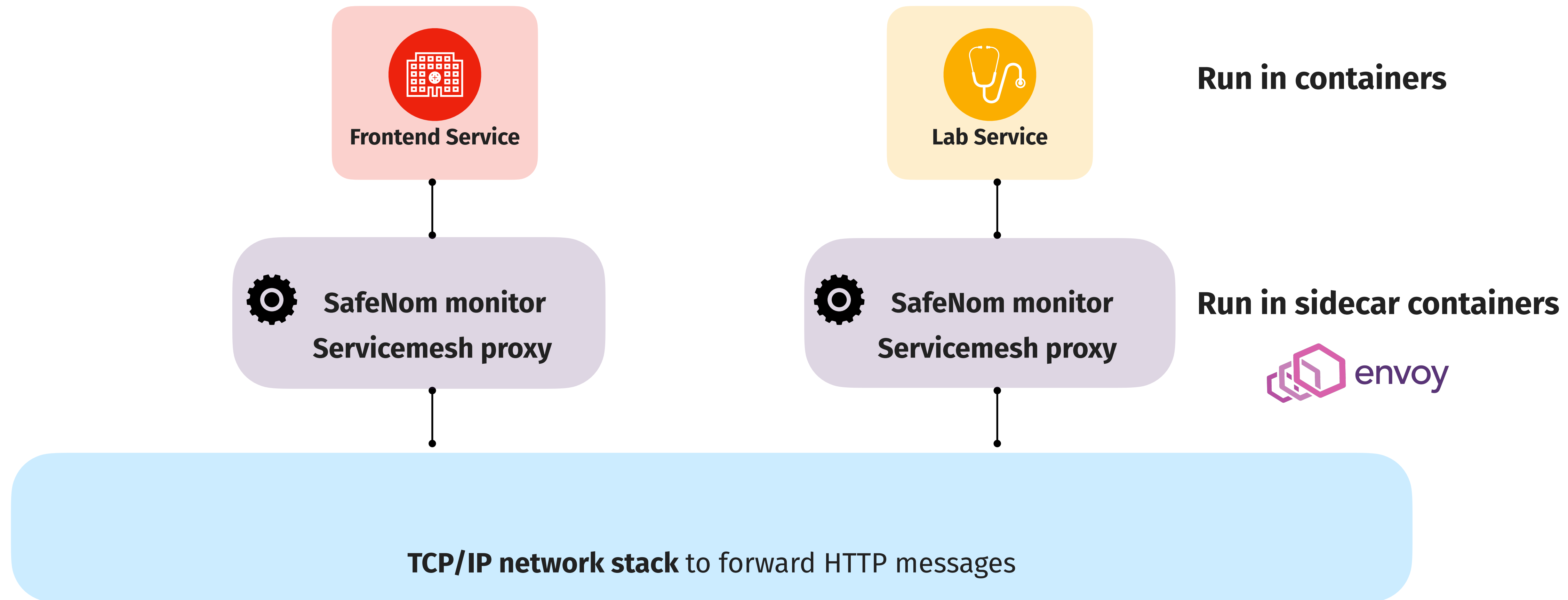
SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK



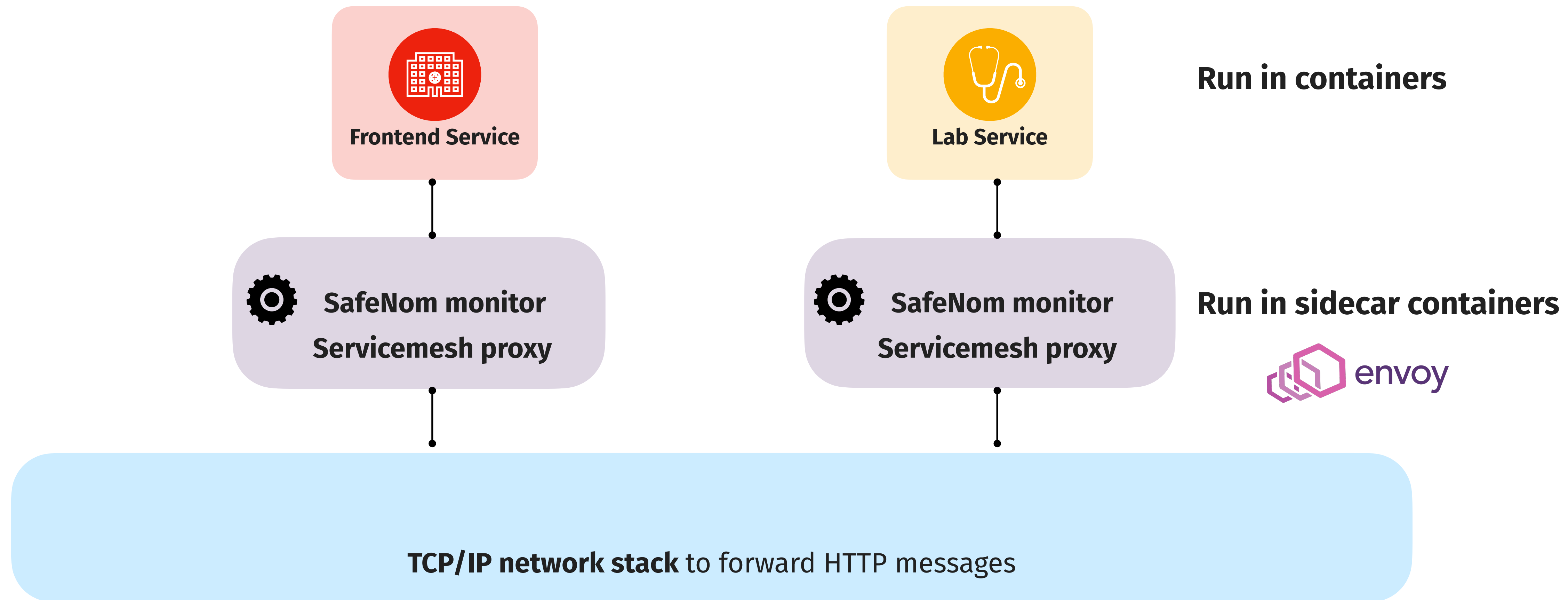
SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK



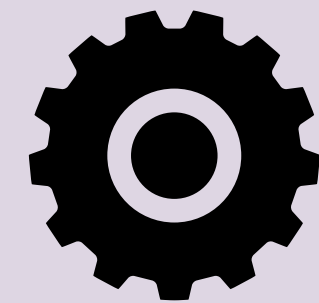
SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK



SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK



SYSTEM ARCHITECTURE OF OUR MONITORING FRAMEWORK



SafeNom monitor
Servicemesh proxy

How is the monitor synthesized?

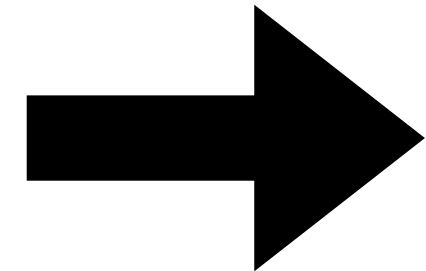
MONITOR IS EXTRACTED FROM THE NOMINAL AUTOMATON



SafeNom monitor
Servicemesh proxy

The NA on the left gives the monitor transitions on the right

NA

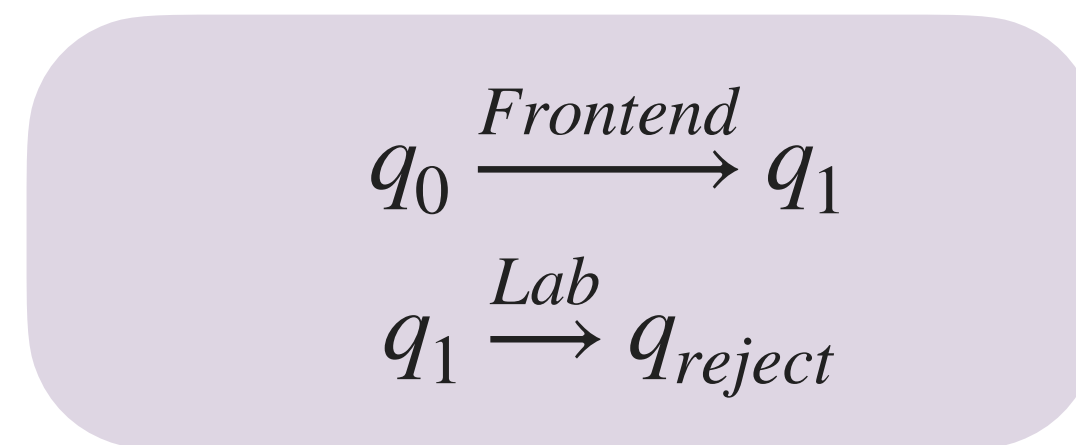
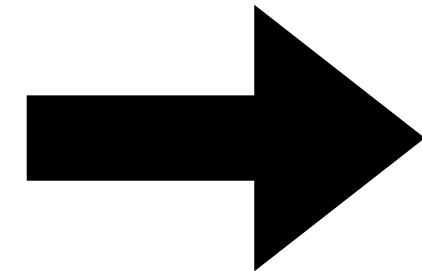
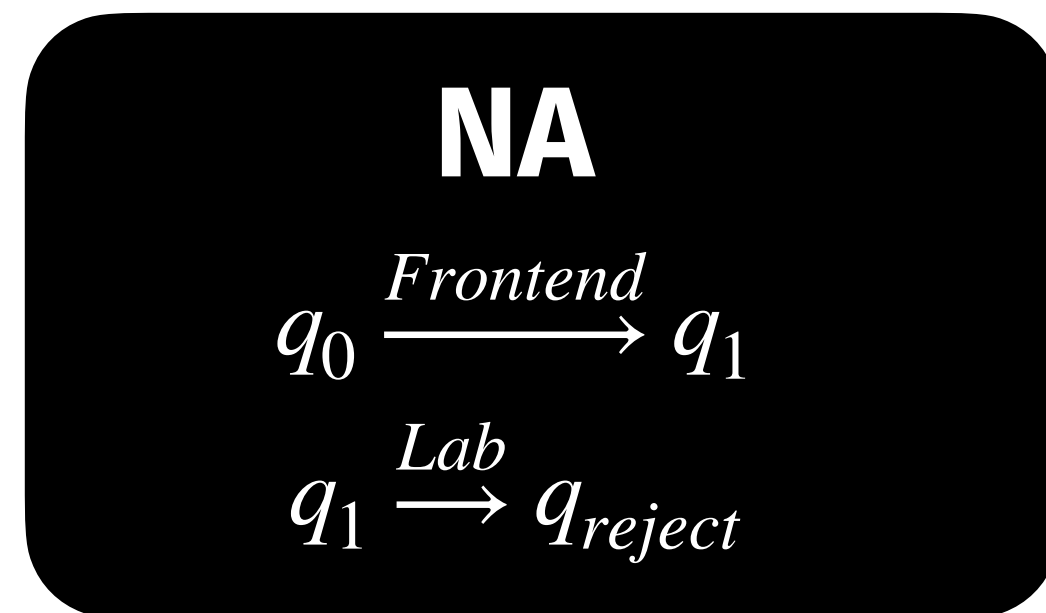
$$q_0 \xrightarrow{\text{Frontend}} q_1$$
$$q_1 \xrightarrow{\text{Lab}} q_{\text{reject}}$$

$$q_0 \xrightarrow{\text{Frontend}} q_1$$
$$q_1 \xrightarrow{\text{Lab}} q_{\text{reject}}$$

MONITOR IS EXTRACTED FROM THE NOMINAL AUTOMATON

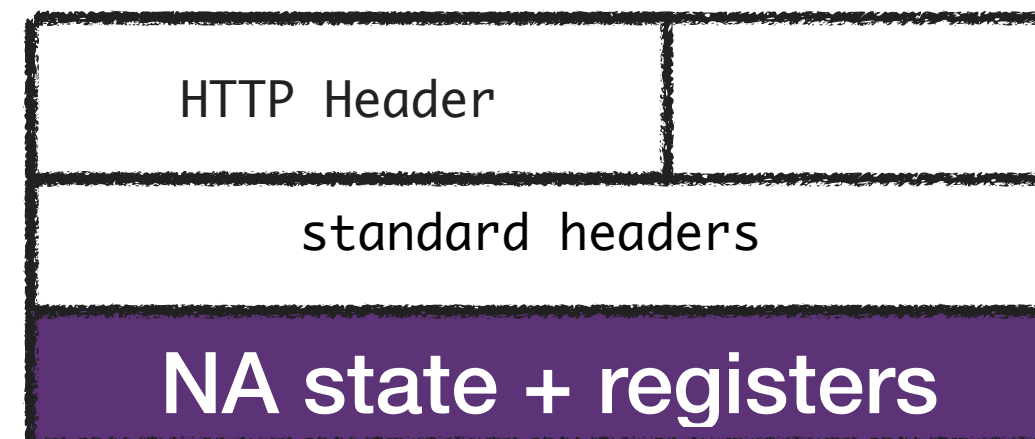


SafeNom monitor
Servicemesh proxy

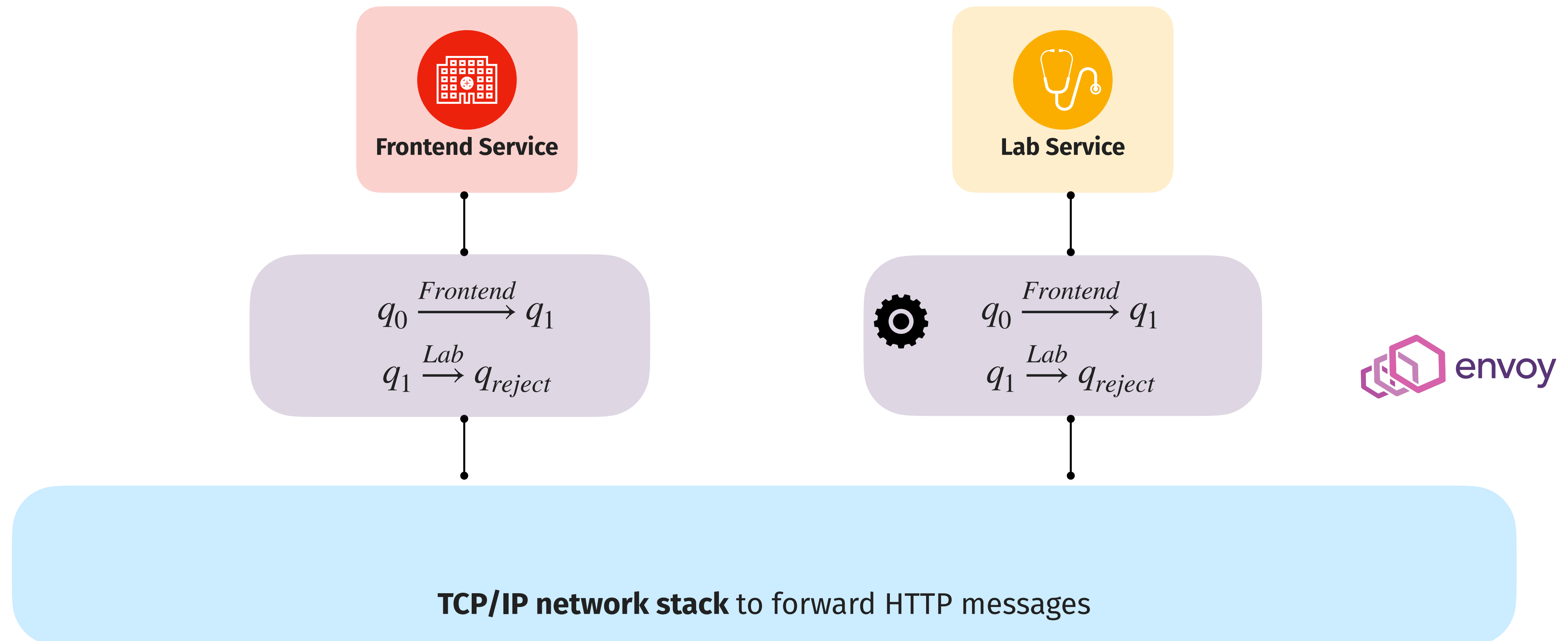
The NA on the left gives the monitor transitions on the right



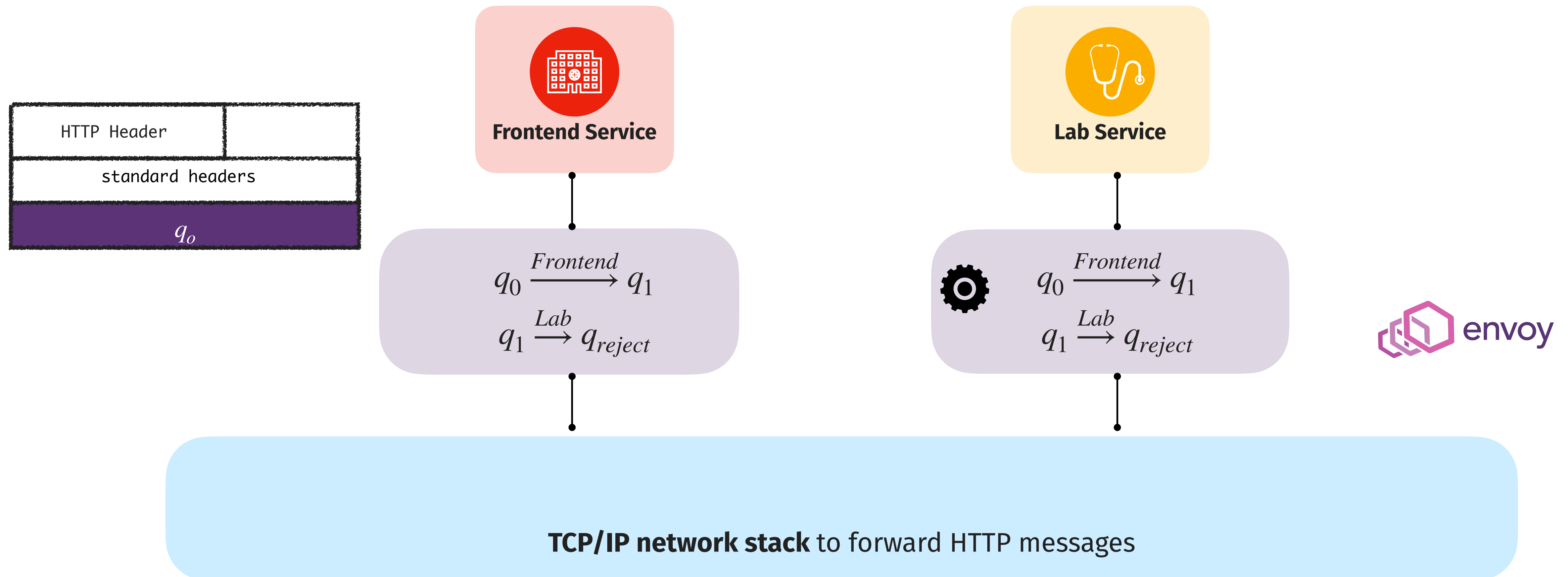
We carry NA configuration in HTTP headers



MONITOR'S DISTRIBUTED RUN

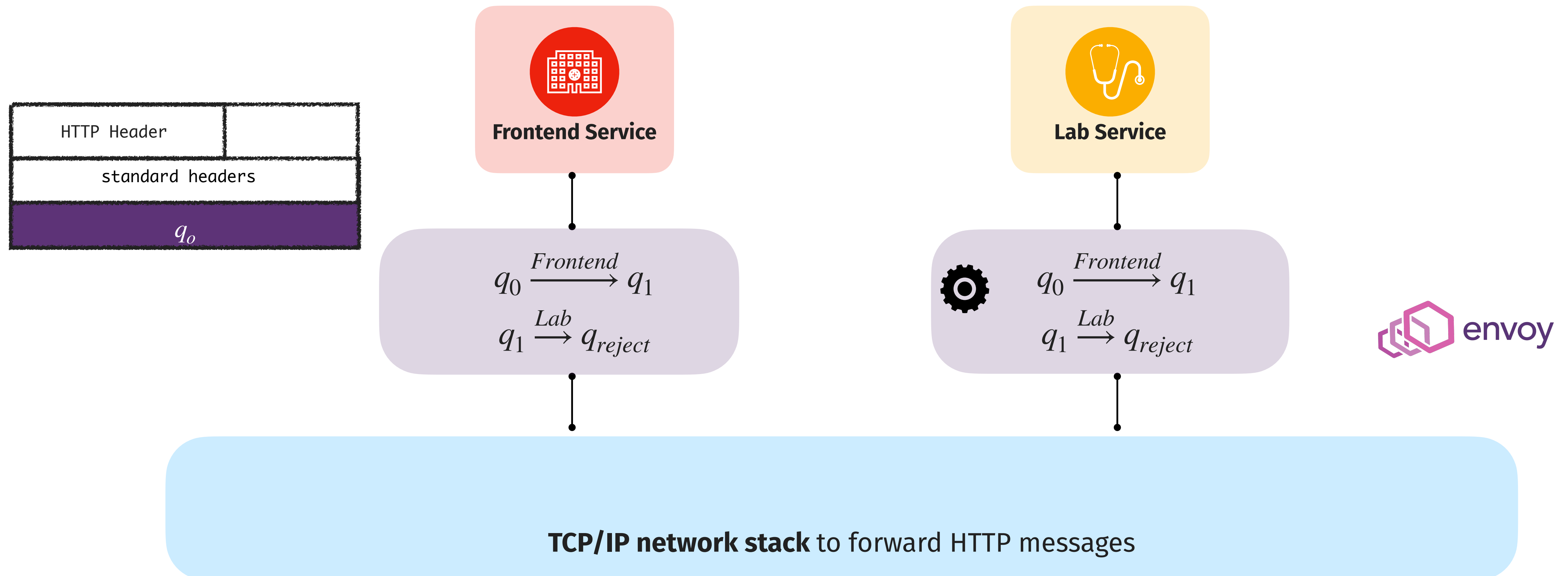


MONITOR'S DISTRIBUTED RUN



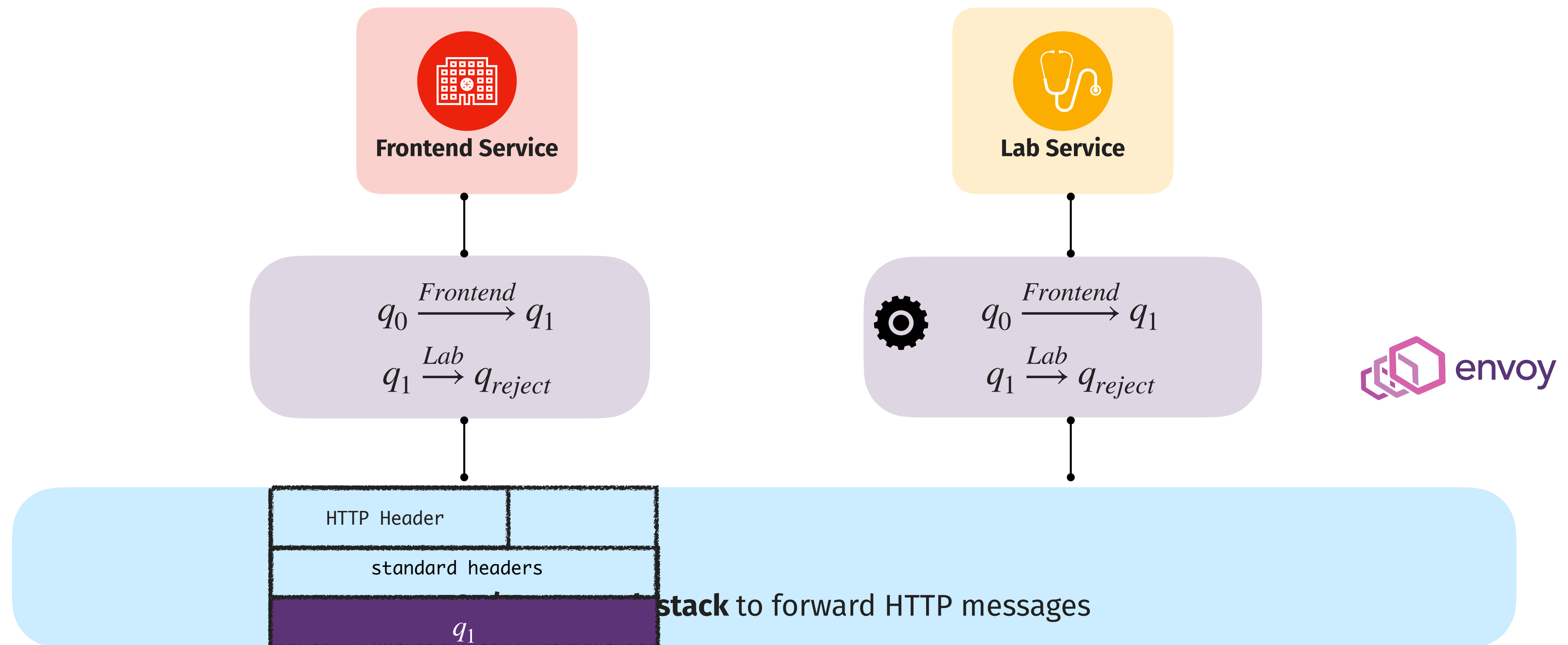
MONITOR'S DISTRIBUTED RUN

Monitor reads the configuration in incoming message; run the NA transition; update the configuration header and allow/block



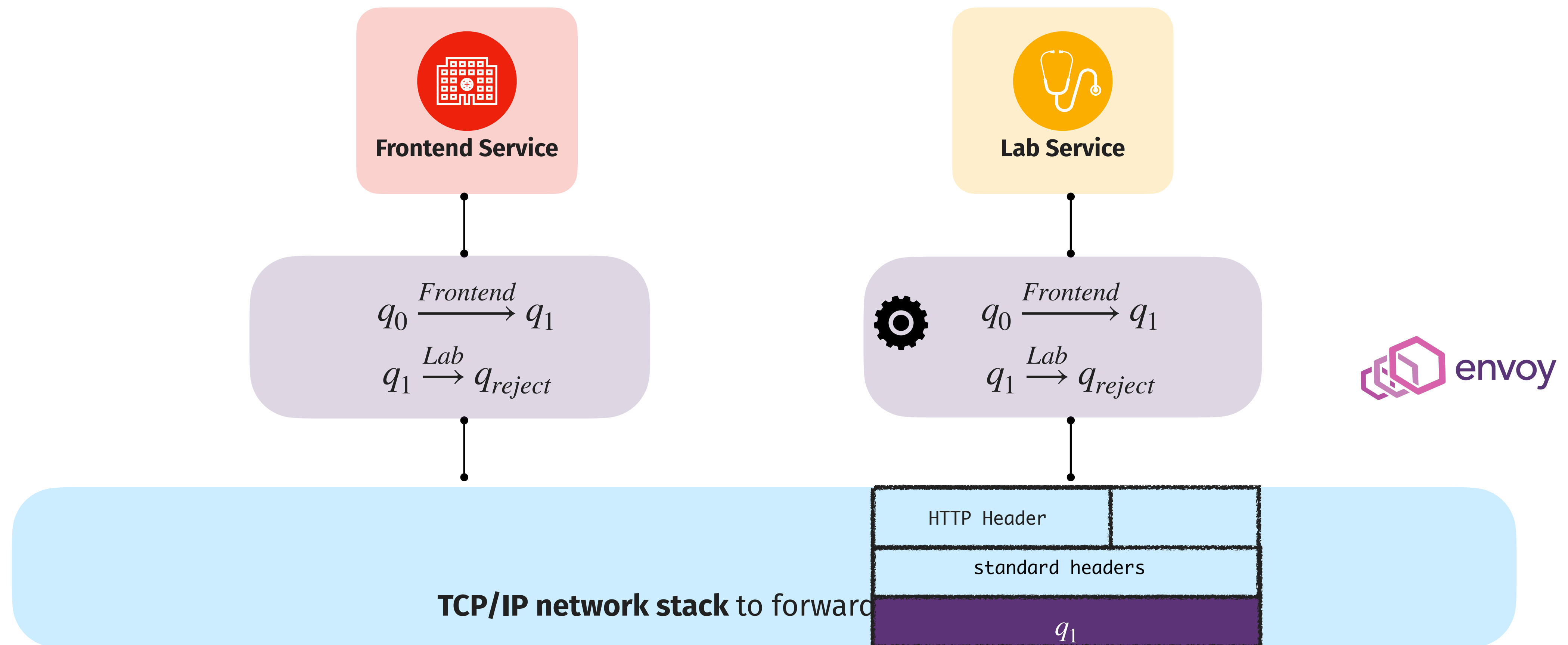
MONITOR'S DISTRIBUTED RUN

Monitor reads the configuration in incoming message; run the NA transition; update the configuration header and allow/block



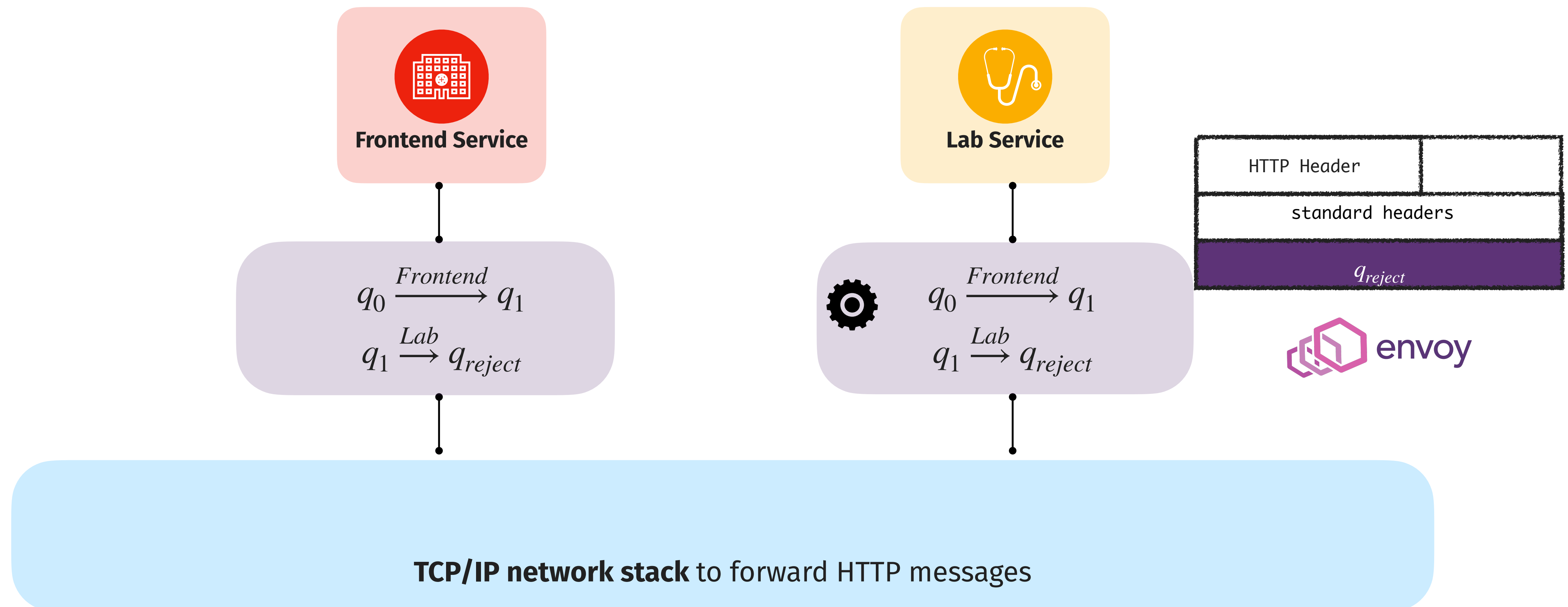
MONITOR'S DISTRIBUTED RUN

Monitor reads the configuration in incoming message; run the NA transition; update the configuration header and allow/block



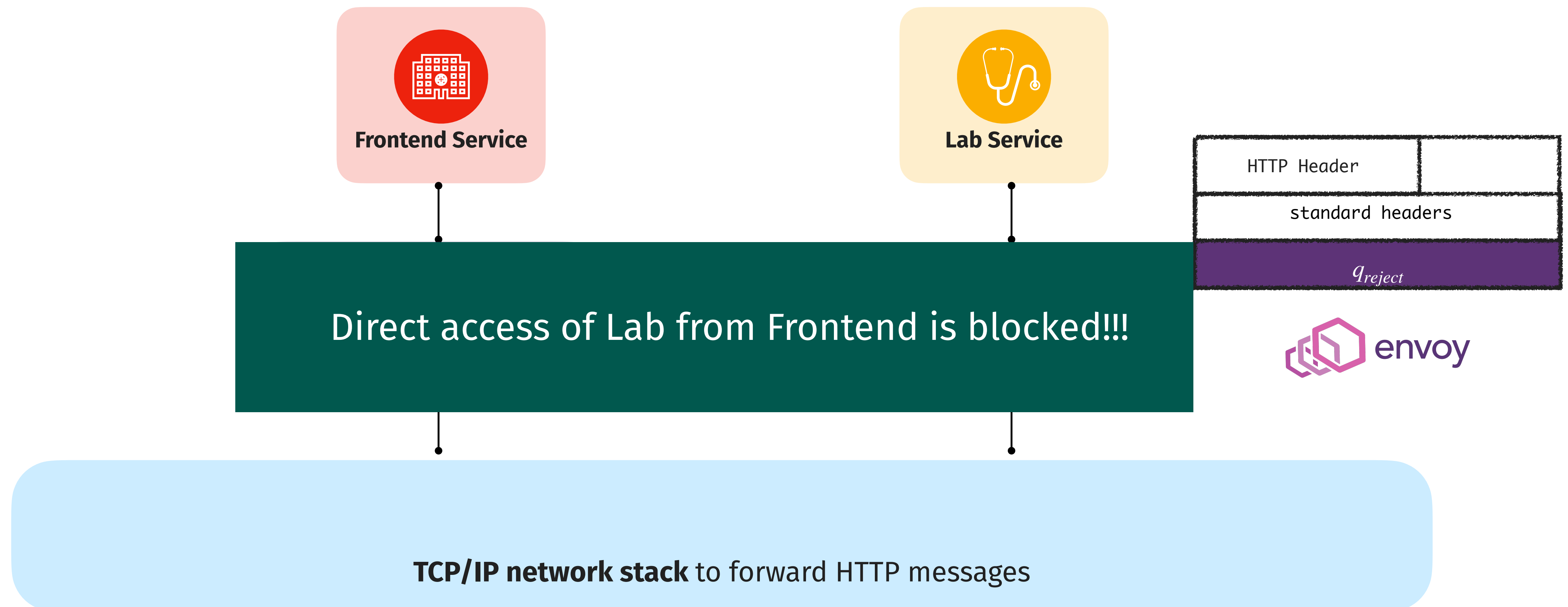
MONITOR'S DISTRIBUTED RUN

Monitor reads the configuration in incoming message; run the NA transition; update the configuration header and allow/block



MONITOR'S DISTRIBUTED RUN

Monitor reads the configuration in incoming message; run the NA transition; update the configuration header and allow/block



AUTOMATON-BASED EXPRESSIVE MONITORING



Key Insight: 1. Data-aware properties as nominal languages
2. Control flow trees as nested words



Policy language

- SafeNom
- SafeTree

Automata-based Distributed Runtime Monitor

- Nominal automata
- Visibly pushdown automata

AUTOMATON-BASED EXPRESSIVE MONITORING



Key Insight: 1. Data-aware properties as nominal languages
2. Control flow trees as nested words



Policy language

- SafeNom
- SafeTree

Automata-based Distributed Runtime Monitor

- Nominal automata
- Visibly pushdown automata

ONGOING WORK...

*Policies to check richer predicates over API parameters
Extension handle asynchronous HTTP communication*

Evaluating SafeTree Monitors

EVALUATION: PERFORMANCE AND MEMORY OVERHEAD

Benchmark microservice applications

9 case studies with varying levels of nesting

<i>Scenario</i>	VPA transitions		VPA states		Latency overhead (ms)	Policy class	Nesting #levels
	#call	#return	#states	#bits			
A/B Testing	6	30	6	3	0.700	Linear	NA
Factorial Testing	11	80	11	4	0.420	Linear	NA
Access Control	12	100	12	4	0.448	Linear	NA
Update	25	544	25	5	0.433	Hierarch.	2
Data-compliance	38	1326	38	6	0.958	Hierarch.	2
Data Proxy	36	1184	36	6	1.117	Hierarch.	3
Encryption	23	454	23	5	0.460	Hierarch.	1
Data Vault	20	346	20	5	0.370	Hierarch.	1
Resource pricing	25	562	25	5	0.457	Hierarch.	2
Hotel Encryption	23	454	23	5	0.216	Hierarch.	1
Hotel Data Proxy	36	1184	36	6	0.443	Hierarch.	3
Hotel Compliance	38	1326	38	6	0.278	Hierarch.	2



EVALUATION: PERFORMANCE AND MEMORY OVERHEAD

1. Latency **overhead** <2ms even for **three leveled policy** with >1k transitions

<i>Scenario</i>	VPA transitions		VPA states		Latency overhead (ms)	Policy class	Nesting #levels
	#call	#return	#states	#bits			
A/B Testing	6	30	6	3	0.700	Linear	NA
Factorial Testing	11	80	11	4	0.420	Linear	NA
Access Control	12	100	12	4	0.448	Linear	NA
Update	25	544	25	5	0.433	Hierarch.	2
Data-compliance	38	1326	38	6	0.958	Hierarch.	2
Data Proxy	36	1184	36	6	1.117	Hierarch.	3
Encryption	23	454	23	5	0.460	Hierarch.	1
Data Vault	20	346	20	5	0.370	Hierarch.	1
Resource pricing	25	562	25	5	0.457	Hierarch.	2
Hotel Encryption	23	454	23	5	0.216	Hierarch.	1
Hotel Data Proxy	36	1184	36	6	0.443	Hierarch.	3
Hotel Compliance	38	1326	38	6	0.278	Hierarch.	2

EVALUATION: PERFORMANCE AND MEMORY OVERHEAD

1. Latency **overhead** <2ms even for **three leveled policy** with >1k transitions

<i>Scenario</i>	VPA transitions		VPA states		Latency overhead (ms)	Policy class	Nesting #levels
	#call	#return	#states	#bits			
A/B Testing	6	30	6	3	0.700	Linear	NA
Factorial Testing	11	80	11	4	0.420	Linear	NA
Access Control	12	100	12	4	0.448	Linear	NA
Update	25	544	25	5	0.433	Hierarch.	2
Data-compliance	38	1326	38	6	0.958	Hierarch.	2
Data Proxy	36	1184	36	6	1.117	Hierarch.	3
Encryption	23	454	23	5	0.460	Hierarch.	1
Data Vault	20	346	20	5	0.370	Hierarch.	1
Resource pricing	25	562	25	5	0.457	Hierarch.	2
Hotel Encryption	23	454	23	5	0.216	Hierarch.	1
Hotel Data Proxy	36	1184	36	6	0.443	Hierarch.	3
Hotel Compliance	38	1326	38	6	0.278	Hierarch.	2

EVALUATION: PERFORMANCE AND MEMORY OVERHEAD

1. Latency **overhead** <2ms even for **three leveled policy** with >1k transitions

Scenario	VPA transitions		VPA states		Latency overhead (ms)	Policy class	Nesting #levels
	#call	#return	#states	#bits			
A/B Testing	6	30	6	3	0.700	Linear	NA
Factorial Testing	11	80	11	4	0.420	Linear	NA
Access Control	12	100	12	4	0.448	Linear	NA
Update	25	544	25	5	0.433	Hierarch.	2
Data-compliance	38	1326	38	6	0.958	Hierarch.	2
Data Proxy	36	1184	36	6	1.117	Hierarch.	3
Encryption	23	454	23	5	0.460	Hierarch.	1
Data Vault	20	346	20	5	0.370	Hierarch.	1
Resource pricing	25	562	25	5	0.457	Hierarch.	2
Hotel Encryption	23	454	23	5	0.216	Hierarch.	1
Hotel Data Proxy	36	1184	36	6	0.443	Hierarch.	3
Hotel Compliance	38	1326	38	6	0.278	Hierarch.	2

EVALUATION: PERFORMANCE AND MEMORY OVERHEAD

2. *Memory overhead due to context header is minimal (≤ 6 bits of stack)*

<i>Scenario</i>	VPA transitions		VPA states		Latency overhead (ms)	Policy class	Nesting #levels
	#call	#return	#states	#bits			
A/B Testing	6	30	6	3	0.700	Linear	NA
Factorial Testing	11	80	11	4	0.420	Linear	NA
Access Control	12	100	12	4	0.448	Linear	NA
Update	25	544	25	5	0.433	Hierarch.	2
Data-compliance	38	1326	38	6	0.958	Hierarch.	2
Data Proxy	36	1184	36	6	1.117	Hierarch.	3
Encryption	23	454	23	5	0.460	Hierarch.	1
Data Vault	20	346	20	5	0.370	Hierarch.	1
Resource pricing	25	562	25	5	0.457	Hierarch.	2
Hotel Encryption	23	454	23	5	0.216	Hierarch.	1
Hotel Data Proxy	36	1184	36	6	0.443	Hierarch.	3
Hotel Compliance	38	1326	38	6	0.278	Hierarch.	2

EVALUATION: PERFORMANCE AND MEMORY OVERHEAD

2. *Memory overhead due to context header is minimal (≤ 6 bits of stack)*

<i>Scenario</i>	VPA transitions		VPA states		Latency overhead (ms)	Policy class	Nesting #levels
	#call	#return	#states	#bits			
A/B Testing	6	30	6	3	0.700	Linear	NA
Factorial Testing	11	80	11	4	0.420	Linear	NA
Access Control	12	100	12	4	0.448	Linear	NA
Update	25	544	25	5	0.433	Hierarch.	2
Data-compliance	38	1326	38	6	0.958	Hierarch.	2
Data Proxy	36	1184	36	6	1.117	Hierarch.	3
Encryption	23	454	23	5	0.460	Hierarch.	1
Data Vault	20	346	20	5	0.370	Hierarch.	1
Resource pricing	25	562	25	5	0.457	Hierarch.	2
Hotel Encryption	23	454	23	5	0.216	Hierarch.	1
Hotel Data Proxy	36	1184	36	6	0.443	Hierarch.	3
Hotel Compliance	38	1326	38	6	0.278	Hierarch.	2